LAPORAN TEKNOLOGI DAN PEMROGRAMAN MOBILE
APLIKASI OPENRENT



Oleh:

Levi Rizki Saputra (123230127)
Muhammad Adel Harits Saifullah (123230156)

Program Studi Informatika
Jurusan Teknik Informatika
Fakultas Teknik Industri
Universitas Pembangunan Nasional "Veteran" Yogyakarta
2026

HALAMAN PENGESAHAN
LAPORAN PROYEK AKHIR

Disusun oleh :

Levi Rizki Saputra	123230127
Muhammad Adel Harits Saifullah	123230156

Telah Diperiksa dan Disetujui oleh Dosen Mata Kuliah

Pada Tanggal : ……

Dosen Teknologi dan Pemrograman Mobile,

Bagus Muhammad Akbar, S. ST., M. Kom.

NIP. 198908012019031013

KATA PENGANTAR

Puji syukur kami panjatkan kehadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya yang tak terhingga, sehingga kami dapat menyelesaikan tugas akhir Teknologi dan Pemrograman Mobile dengan baik, serta menyusun laporan tugas akhir ini sebagai bentuk pertanggungjawaban akademik. Laporan ini merupakan hasil dari proses pembelajaran dan implementasi praktis yang telah kami laksanakan selama semester ini.

Kami juga mengucapkan terima kasih yang sebesar-besarnya kepada Bapak Bagus Muhammad Akbar S.ST., M.Kom. selaku dosen pengampu mata kuliah Teknologi dan Pemrograman Mobile yang senantiasa membimbing, memberikan arahan, dan membagikan ilmu pengetahuan dengan penuh kesabaran. Bimbingan beliau sangat berarti dalam membantu kami memahami konsep-konsep kompleks dan menerapkan solusi teknis yang diperlukan dalam proyek ini. Dedikasi Bapak dosen telah menjadi inspirasi bagi kami untuk terus belajar dan berinovasi.

Kami menyadari bahwa laporan ini masih memiliki banyak kekurangan dan jauh dari kata sempurna, baik dari segi kedalaman materi, tata bahasa, maupun penyajian. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari berbagai pihak guna penyempurnaan laporan ini di masa mendatang. Masukan yang konstruktif akan sangat berharga bagi kami untuk meningkatkan kualitas pekerjaan kami.

Terima kasih pula kami sampaikan kepada semua pihak yang telah memberikan dukungan, fasilitas, dan bantuan dalam bentuk apapun selama proses penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat yang maksimal bagi pembaca, khususnya sebagai referensi dan tambahan wawasan dalam bidang Kriptografi. Kami berharap laporan ini dapat dipergunakan sebagaimana mestinya dan memberikan kontribusi positif bagi pengembangan ilmu pengetahuan.

Yogyakarta, 4 Mei 2026

Penyusun 1

Levi Rizki Saputra

NIM. 123230127

Penyusun 2

M. Adel Harits Saifullah

NIM. 123230156

DAFTAR ISI

1 Pendahuluan 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan 1
1.4 Manfaat 1
1.4.1 Bagi Penyewa 1
1.4.2 Bagi Pemilik Produk 2
1.4.3 Bagi Pengembang 2
1.5 Batasan Masalah 2
2 Analisis dan Perancangan Sistem 3
2.1 Analisis Kebutuhan Sistem 3
2.1.1 Kebutuhan Fungsional 3
2.1.2 Kebutuhan Non-Fungsional 4
2.2 Perancangan Sistem 5
2.2.1 Use Case Diagram 5
2.2.2 Activity Diagram 6
2.2.3 Sequence Diagram 13
2.3 Perancangan Arsitektur Sistem 20
2.3.1 Arsitektur Client (Mobile) 20
2.3.2 Arsitektur Server (Backend) 20
2.3.3 Teknologi yang Digunakan 20
2.4 Perancangan Database 21
2.4.1 ERD Database Server 21
2.4.2 Rancangan Database Client 22
3 Implementasi dan Pembahasan 23
3.1 Lingkungan Pengembangan 23
3.2 Implementasi Backend 23
3.2.1 Struktur Direktori Backend 23
3.2.2 Implementasi Autentikasi 23
3.2.3 Implementasi API Endpoint 24
3.2.4 Login dan Register 26
3.2.5 JWT 28
3.2.6 Pencarian Berdasarkan Stok, Kata Kunci dan AI Search 31
3.2.7 Pengiriman Notifikasi Ke Perangkat Pengguna 32
3.2.8 Penghitungan Skor Review Berdasarkan Skor Relevansi AI 34
3.2.9 Pengujian Stok Pada Permintaan Sewa 35
3.3 Implementasi Frontend (Mobile) 36

3.3.1 Struktur Direktori Client 36
3.3.2 JWT Interceptor 36
3.3.3 Autentikasi 40
3.3.4 Mencari Alamat Terdekat dengan Pengguna 42
3.3.5 Cek Stok di Tanggal atau Renttang Tanggal 43
3.3.6 Mini Game 44
3.4 Tampilan Antarmuka 58
4 Pengujian Sistem 65
4.1 Metodologi Pengujian 65
4.2 Rencana Pengujian 65
4.2.1 Modul Autentikasi 65
4.2.2 Modul Alamat 66
4.2.3 Modul Produk 66
4.2.4 Modul Penyewaan 68
4.2.5 Modul Chat 69
4.2.6 Modul Profil dan Pengaturan 70
4.2.7 Modul Mini Game 70
4.3 Hasil Pengujian 70
4.3.1 Rekapitulasi Hasil Pengujian 70
4.3.2 Analisis Hasil Pengujian 71
5 Penutup 72
5.1 Kesimpulan 72
5.2 Saran 72
5.2.1 Peningkatan Fitur 72
5.2.2 Peningkatan Teknis 72

1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi yang pesat telah membawa perubahan signifikan dalam berbagai aspek kehidupan, termasuk dalam sektor properti dan penyewaan tempat tinggal. Kebutuhan masyarakat akan kemudahan akses informasi mengenai properti sewaan semakin meningkat seiring dengan pertumbuhan jumlah penduduk urban dan mobilitas yang tinggi.

Aplikasi OpenRent hadir sebagai solusi digital berbasis mobile yang menjembatani kebutuhan antara pemilik produk dengan calon penyewa. Dengan memanfaatkan teknologi pemrograman mobile modern, OpenRent memungkinkan pengguna untuk mencari, menampilkan, dan menyewa produk secara mudah, cepat, dan efisien langsung dari perangkat smartphone mereka.

Mata kuliah Teknologi dan Pemrograman Mobile memberikan kesempatan bagi mahasiswa untuk mengimplementasikan konsep-konsep pengembangan aplikasi mobile secara nyata. Laporan ini mendokumentasikan proses perancangan, pengembangan, dan pengujian aplikasi OpenRent sebagai proyek akhir mata kuliah tersebut.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam pengembangan aplikasi OpenRent adalah sebagai berikut:

1. Bagaimana merancang dan membangun aplikasi mobile yang memudahkan proses pencarian dan penyewaan produk?
2. Bagaimana mengimplementasikan sistem autentikasi yang aman bagi pengguna aplikasi?
3. Bagaimana mengintegrasikan fitur manajemen produk bagi pemilik dengan fitur pencarian bagi penyewa dalam satu platform mobile?
4. Bagaimana membangun antarmuka pengguna (UI) yang intuitif dan responsif pada platform mobile?

1.3 Tujuan

Tujuan dari pengembangan aplikasi OpenRent adalah:

1. Membangun aplikasi mobile OpenRent yang memfasilitasi proses pencarian dan penyewaan produk secara digital.
2. Mengimplementasikan fitur autentikasi pengguna dengan sistem yang aman dan handal.
3. Menyediakan fitur manajemen produk yang lengkap bagi pemilik dan fitur pencarian yang informatif bagi calon penyewa.
4. Menghasilkan antarmuka pengguna yang modern, responsif, dan mudah digunakan.

1.4 Manfaat

Manfaat yang diharapkan dari pengembangan aplikasi OpenRent meliputi:

1.4.1 Bagi Penyewa

- Kemudahan dalam mencari produk sewaan yang sesuai kebutuhan.
- Akses informasi produk yang lengkap dan transparan.
- Proses negosiasi dan pengajuan sewa yang lebih efisien.

1.4.2 Bagi Pemilik Produk

- Platform untuk memasarkan produk sewaan secara digital.
- Kemudahan dalam mengelola informasi dan ketersediaan produk.
- Jangkauan calon penyewa yang lebih luas.

1.4.3 Bagi Pengembang

- Pengalaman praktis dalam pengembangan aplikasi mobile full-stack.
- Penerapan konsep teknologi mobile dalam proyek nyata.

1.5 Batasan Masalah

Dalam pengembangan aplikasi OpenRent, ditetapkan batasan masalah sebagai berikut:

1. Aplikasi dikembangkan untuk platform Android menggunakan Flutter.
2. Backend dikembangkan menggunakan teknologi Go dan menggunakan PostgeeSQL sebagai database.
3. Fitur pembayaran sewa tidak termasuk dalam lingkup pengembangan aplikasi ini.
4. Aplikasi hanya mencakup wilayah Indonesia sebagai target pasar.
5. Pengujian dilakukan pada perangkat Android dengan versi 16 (Baklava).

2 Analisis dan Perancangan Sistem

2.1 Analisis Kebutuhan Sistem

Tahap analisis kebutuhan bertujuan untuk mengidentifikasi kebutuhan fungsional dan non-fungsional yang harus dipenuhi oleh aplikasi OpenRent.

2.1.1 Kebutuhan Fungsional

Kebutuhan fungsional merupakan deskripsi fitur dan layanan yang harus disediakan oleh sistem:

Modul Autentikasi:

- Pengguna dapat mendaftar akun baru sebagai penyewa atau pemilik produk.
- Pengguna dapat masuk (login) menggunakan email dan kata sandi.
- Pengguna dapat mengatur agar diperlukan biometrik sebelum masuk aplikasi
- Pengguna dapat keluar (logout) dari aplikasi.

Modul Alamat:

- Pengguna dapat menambahkan alamat baru
- Pengguna dapat menghapus dan mengubah alamat
- Pengguna dapat melihat daftar alamat

Modul Produk:

- Pemilik dapat menambahkan produk baru beserta detail dan foto.
- Pemilik dapat memilik alamat berdasarkan lokasi saat ini pada saat menambahkkan produk
- Pemilik dapat mengedit produk yang dimiliki.
- Peminjam dapat melihat daftar produk.
- Peminjam dapat melihat detail lengkap suatu produk.
- Peminjam dapat mencari produk berdasarkan kata kunci dan ketersedian stok di rentang tanggal tertentu
- Peminjam dapat melihat daftar review yang telah diurutkan dari tingkat skor relevansi AI
- Peminjam dapat melihat harga estimasi yang telah disesuaikan dengan mata uang pilihan
- Peminjam mendapatkan rekomendasi produk yang mirip

Modul Penyewaan:

- Penyewa dapat menentukan tanggal awal sewa dan akhir sewa
- Penyewa dapat mengajukan permintaan sewa pada suatu produk.
- Pemilik dapat menerima atau menolak permintaan sewa.
- Peminjam dapat mengajukan permintaan pengambilan produk
- Pemilik dapat mengkonfirmasi pengambilan produk dan mencatat pembayaran awal
- Peminjam dapat mengajukan permintaan pengembalian produk
- Pemilik dapat mengkonfimasi pengembalian produk dan mencatatat pembayaran akhir termasuk denda
- Pemilik dapat menlakukan konversi mata uang saat menerima pembayaran
- Pengguna mendapatkan notifikasi jika status penyewaan berubah

- Pengguna dapat melihat riwayat dan status penyewaan.
- Penyewa dapat membuat review produk dari penyewaan yang sudah selesai
- Penyewa dapat melihat estimasi pembayaran berdasarkan mata uang pilihan
- Pengguna dapat melihat waktu penyewaan dalam zona waktu pilihan

Modul Chat:

- Penyewa dan pemilik dapat melakukan komunikasi pesan
- Pengguna mendapatkan notifikasi jika mendapatkan pesan baru

Modul Profil:

- Pengguna dapat mengunggah foto profil.

Modul Pengaturan:

- Pengguna dapat mengatur preferensi aplikasi seperti biometrik, mata uang, dan zona waktu.

Modul Mini Game:

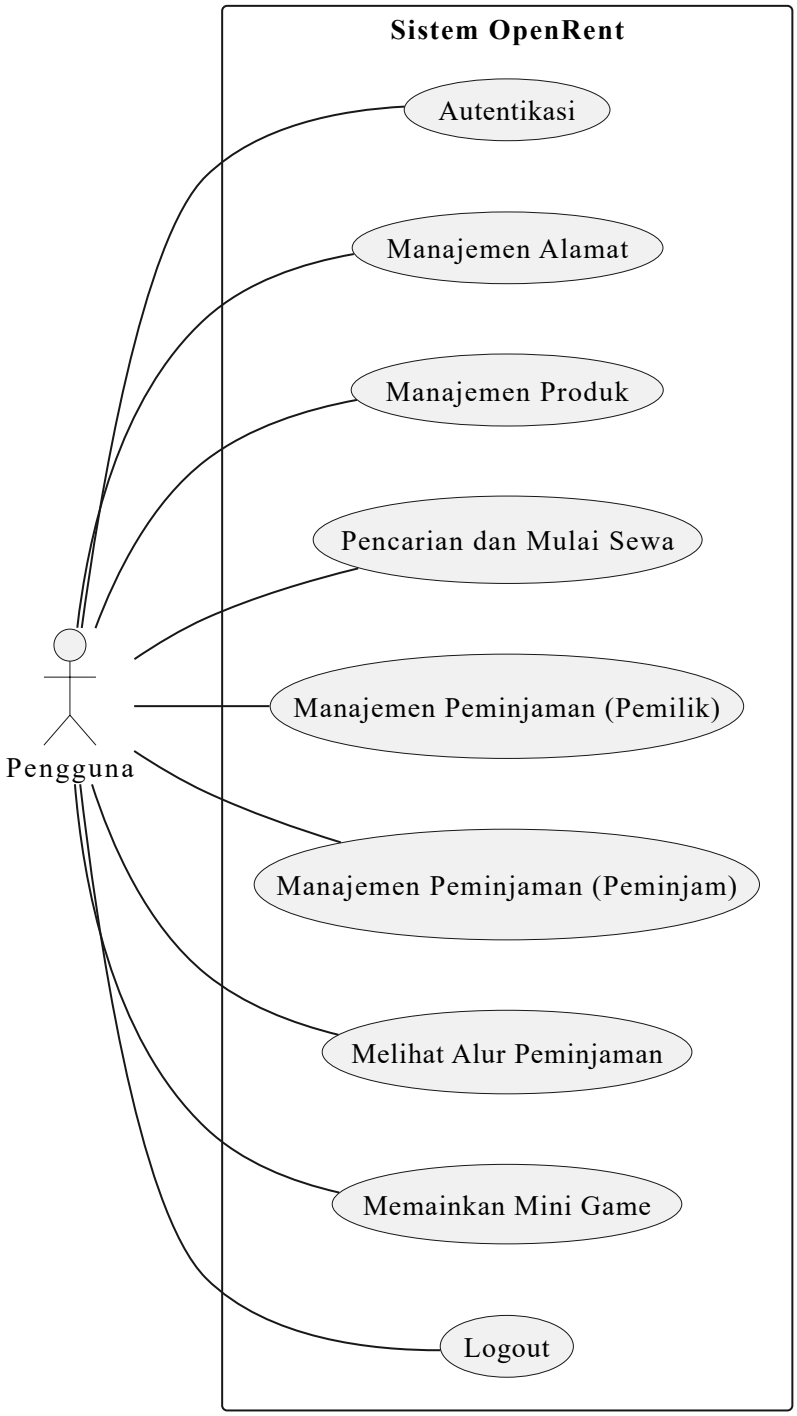
- Pengguna dapat memainkan mini game yang dapat melatih keseimbangan serta menggunakan Giroskop dan Akselerometer

2.1.2 Kebutuhan Non-Fungsional

- **Performa:** Waktu respons aplikasi tidak lebih dari 3 detik pada kondisi jaringan normal.
- **Keamanan:** Data sensitif pengguna dienkripsi; autentikasi menggunakan JWT (JSON Web Token). Password harus disimpan menggunakan algoritma Argon2id.
- **Kegunaan:** Antarmuka intuitif yang dapat dioperasikan tanpa pelatihan khusus.
- **Keandalan:** Sistem backend tersedia minimal 99% waktu operasional.
- **Skalabilitas:** Arsitektur mendukung penambahan fitur di masa mendatang.

2.2 Perancangan Sistem

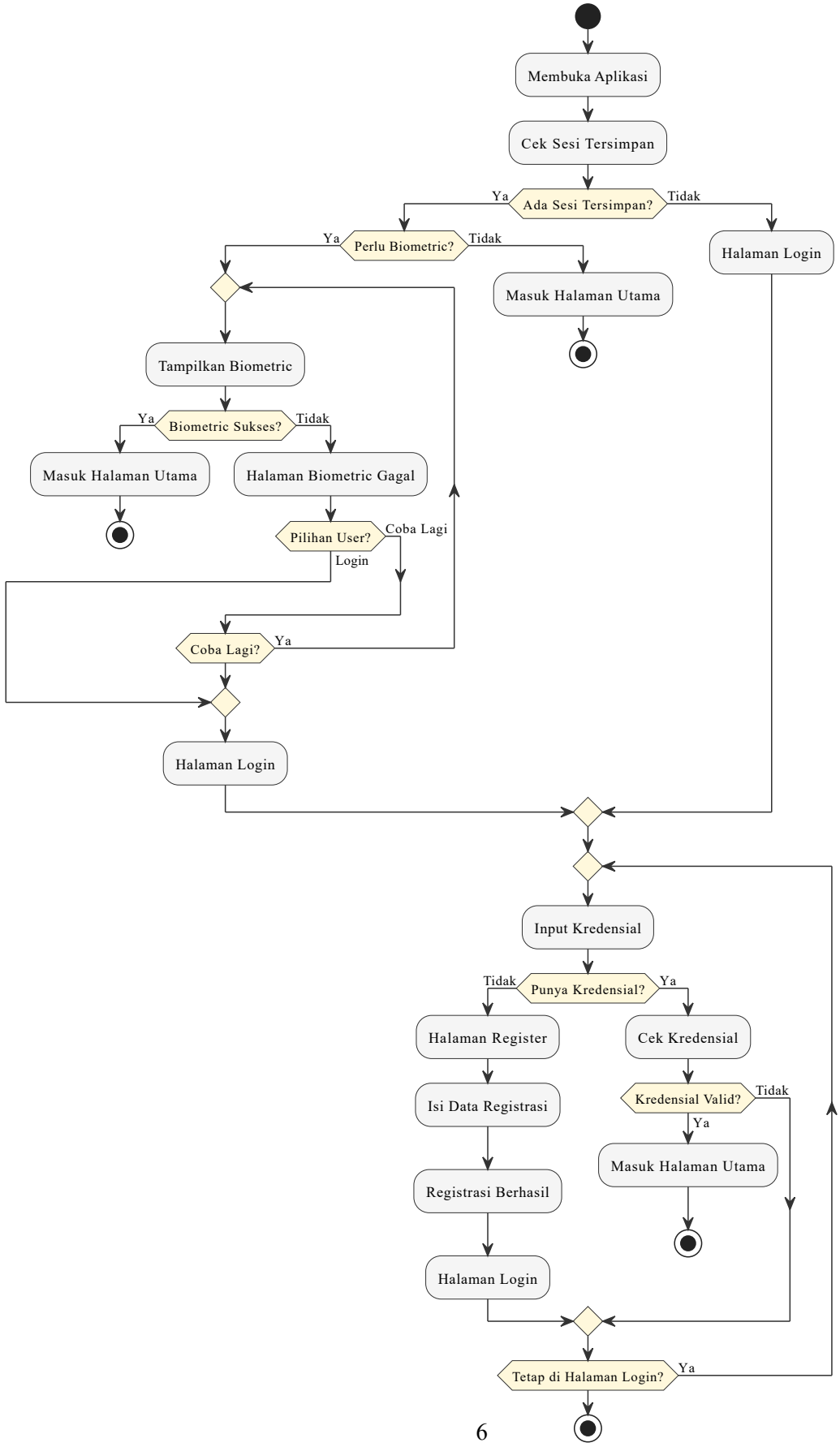
2.2.1 Use Case Diagram



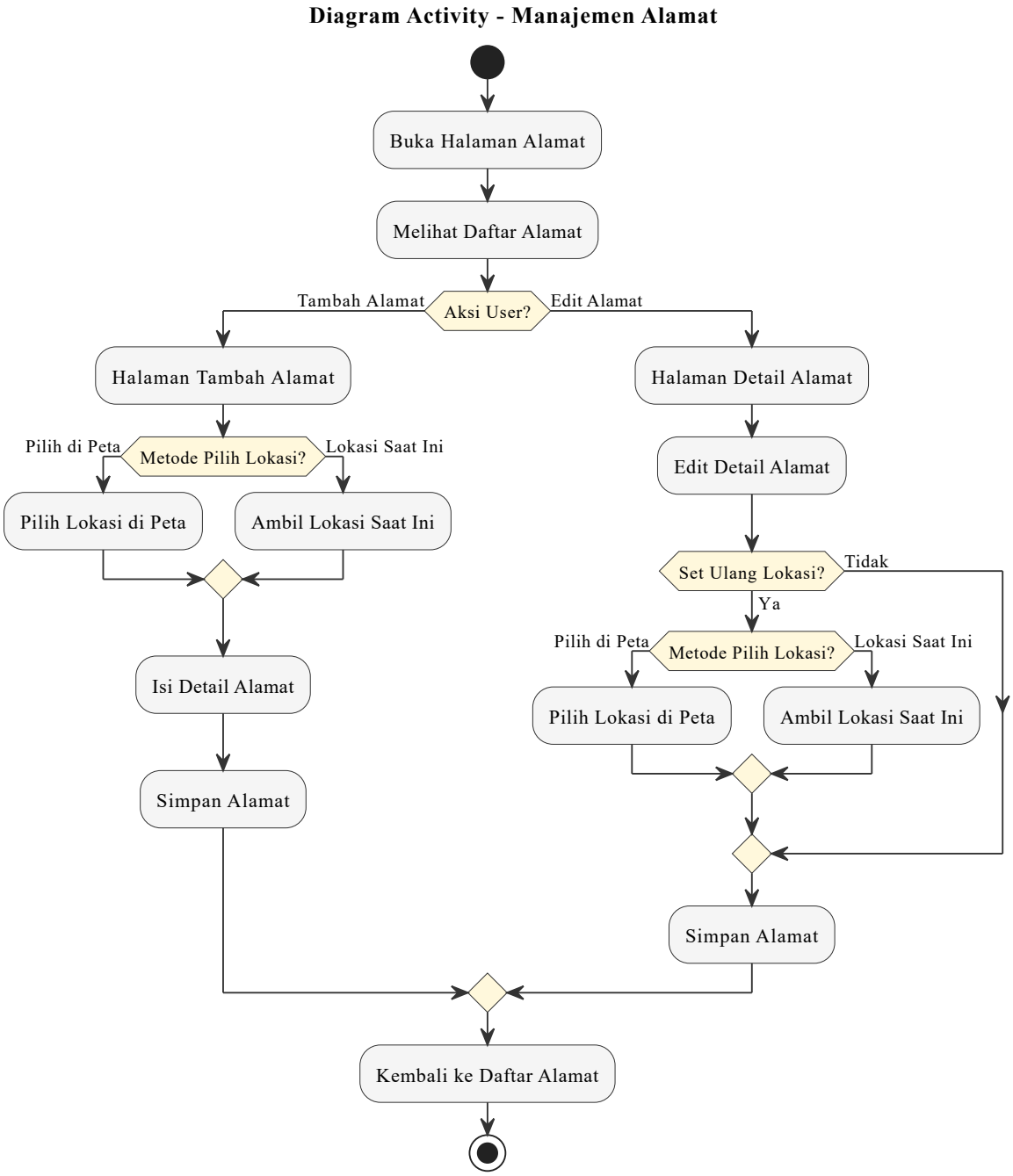
2.2.2 Activity Diagram

A. Autentikasi

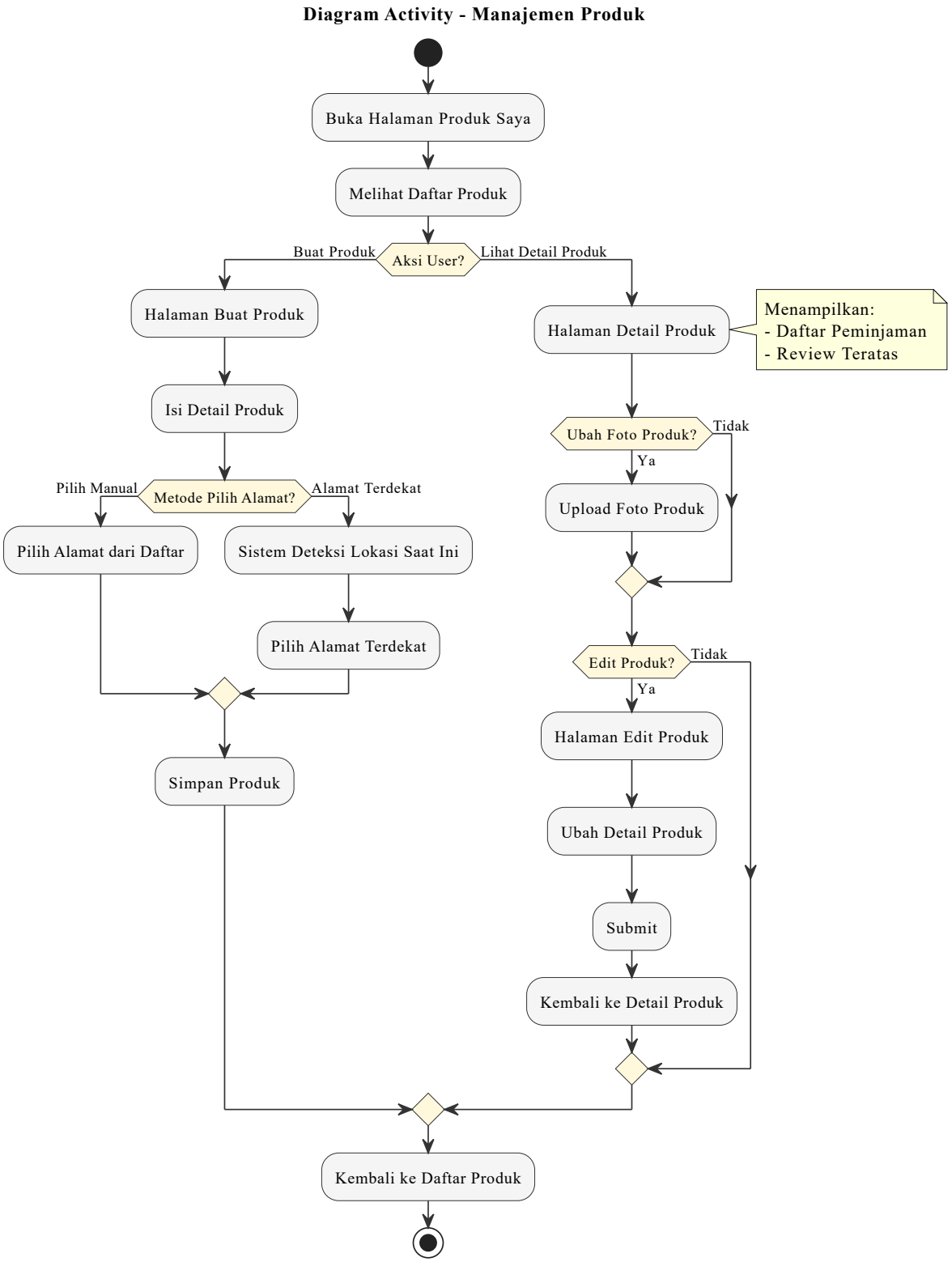
Diagram Activity - Autentikasi



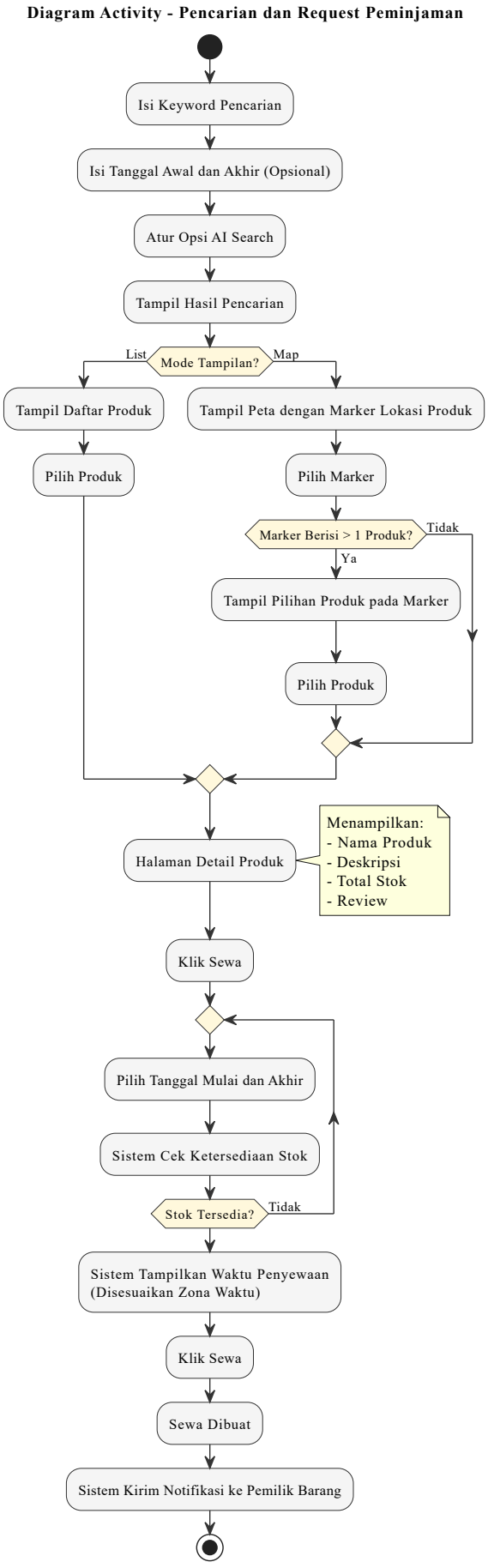
B. Manajemen Alamat



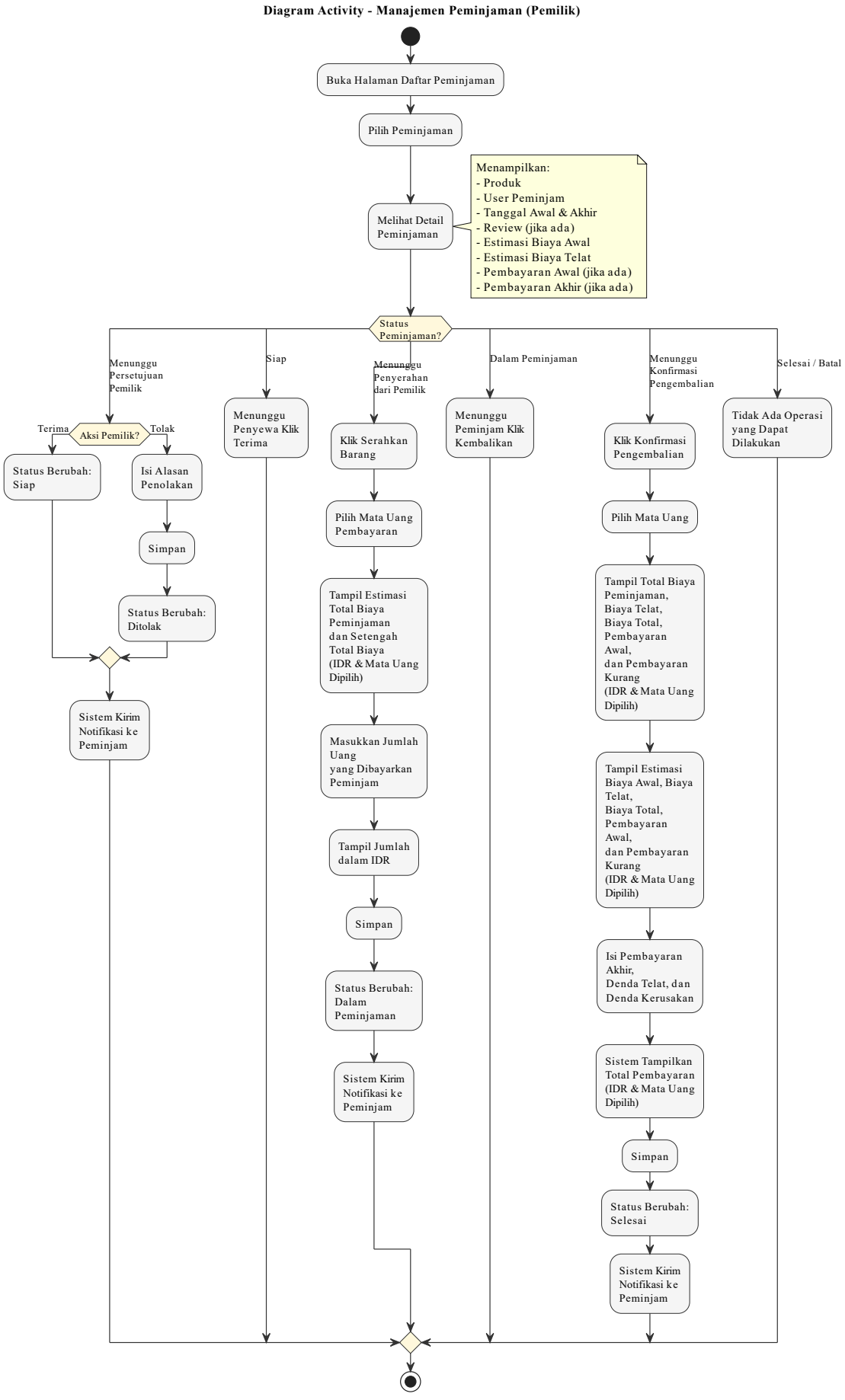
C. Manajemen Produk



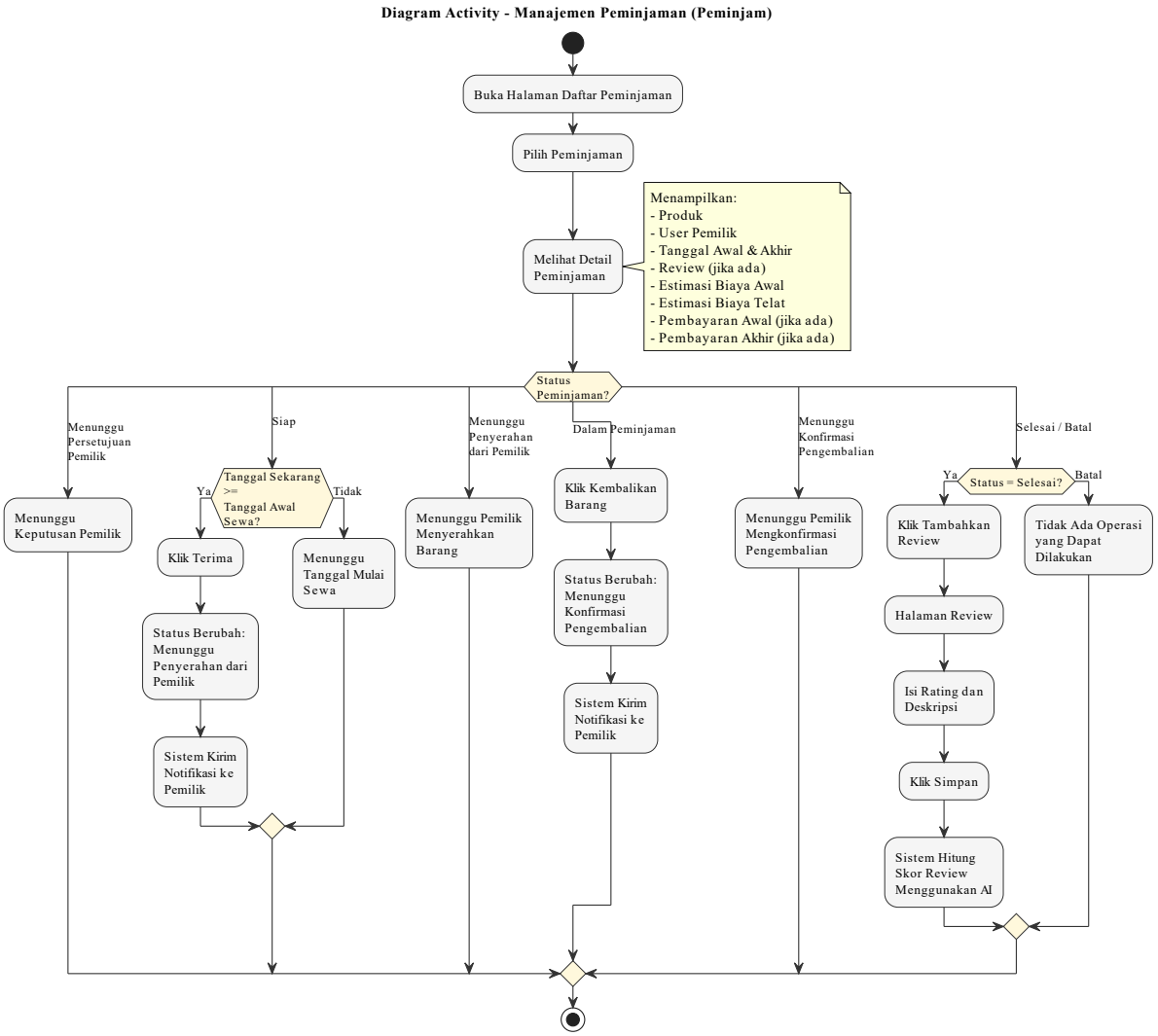
D. Pencarian dan Mulai Sewa



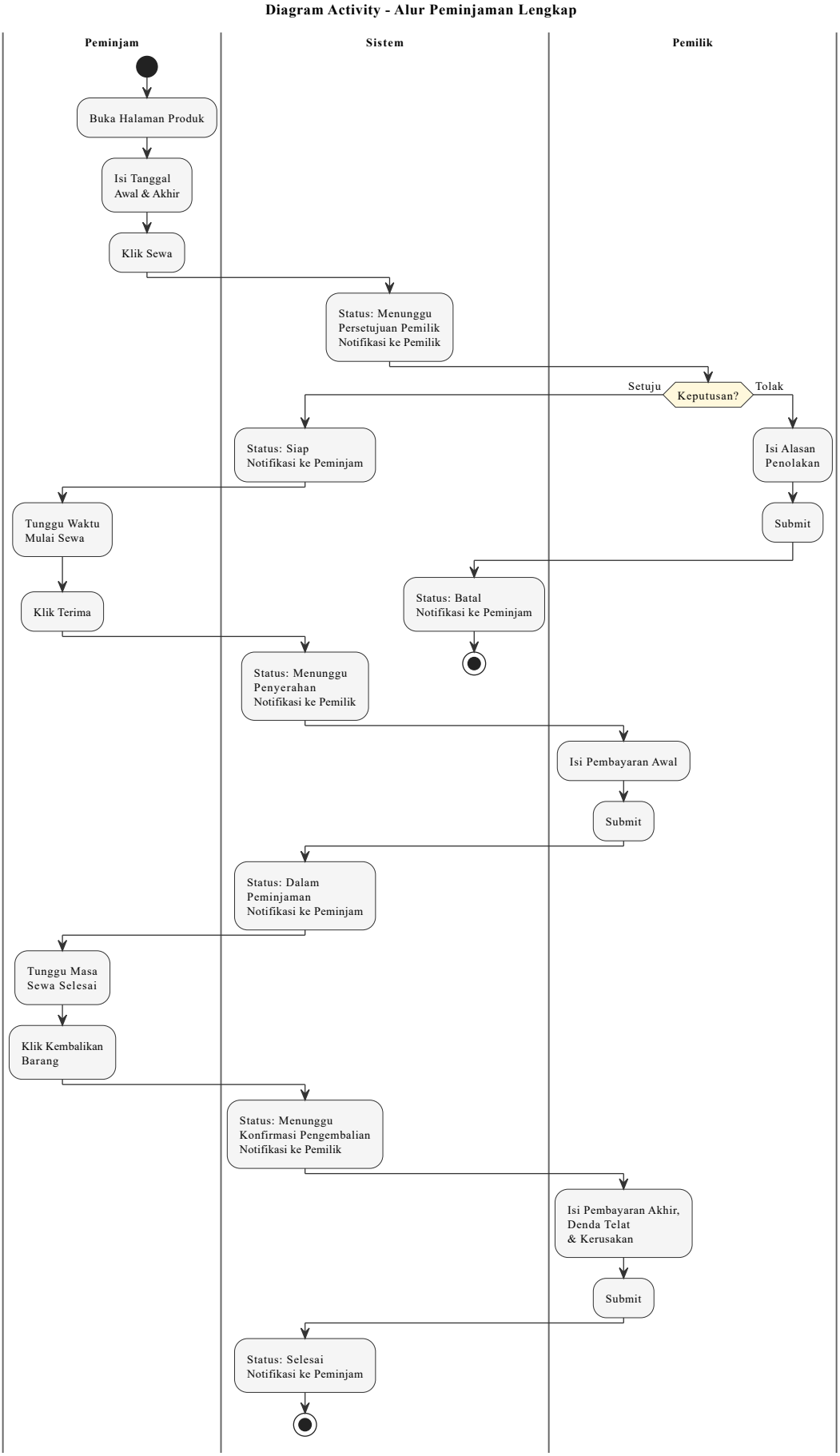
E. Manajemen Peminjaman untuk Pemilik Barang



F. Manajemen Peminjaman untuk Peminjam

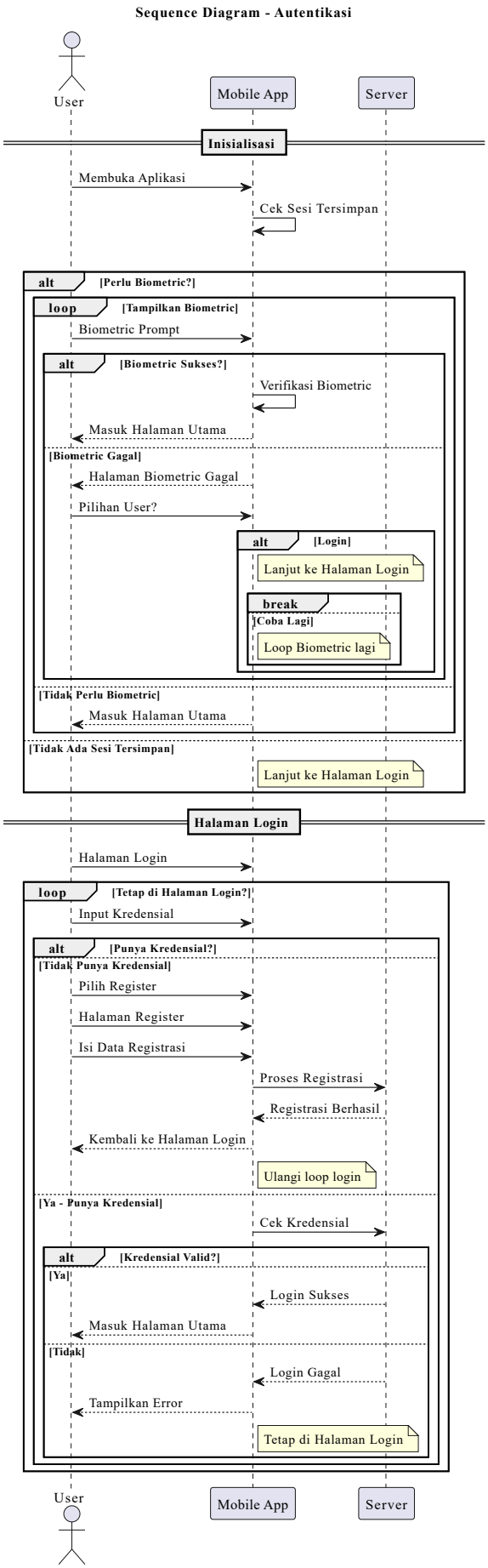


G. Alur Peminjaman Secara Lengkap

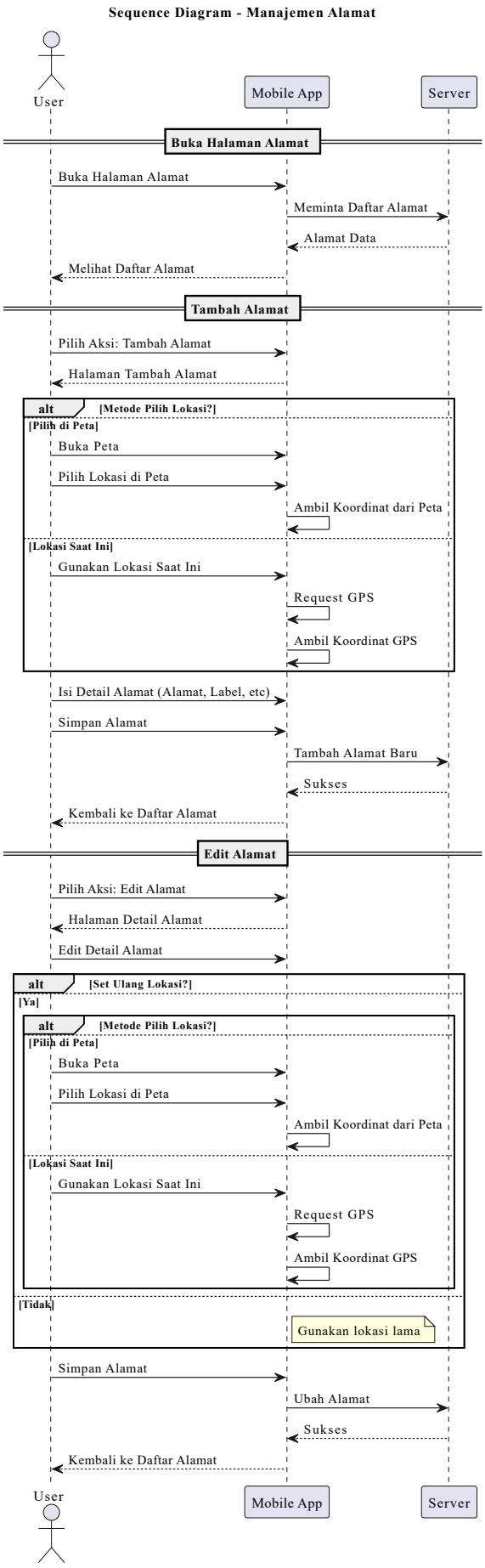


2.2.3 Sequence Diagram

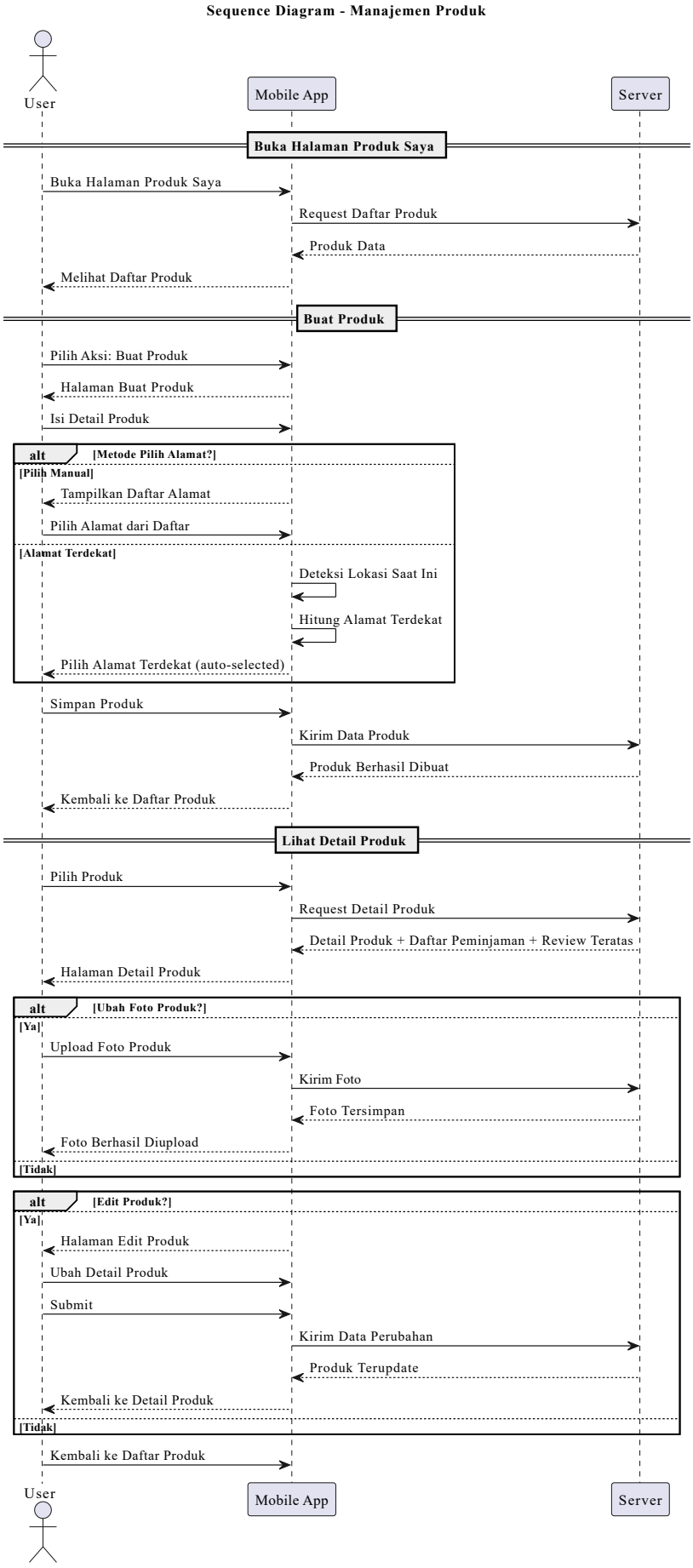
A. Autentikasi



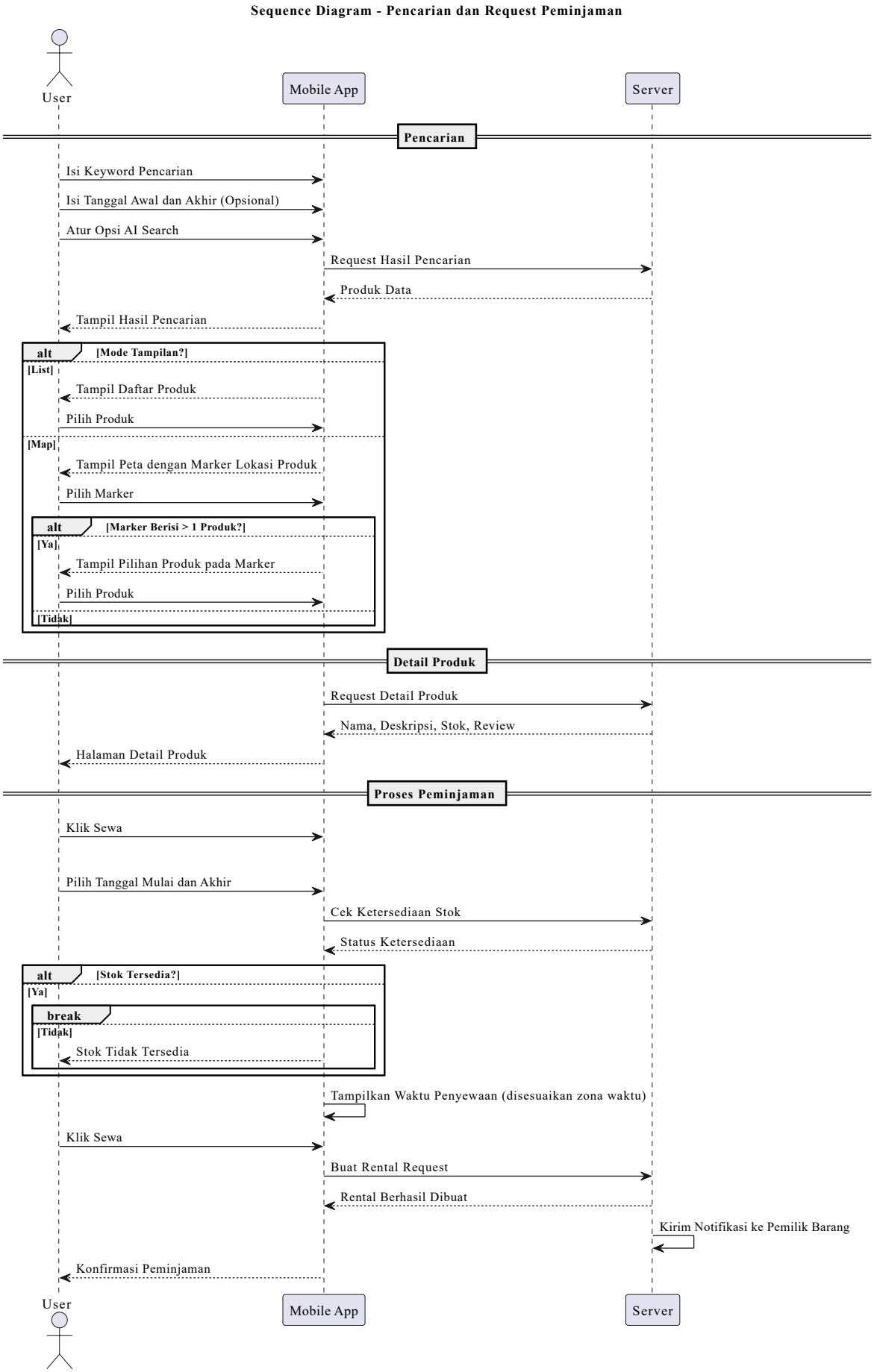
B. Manajemen Alamat



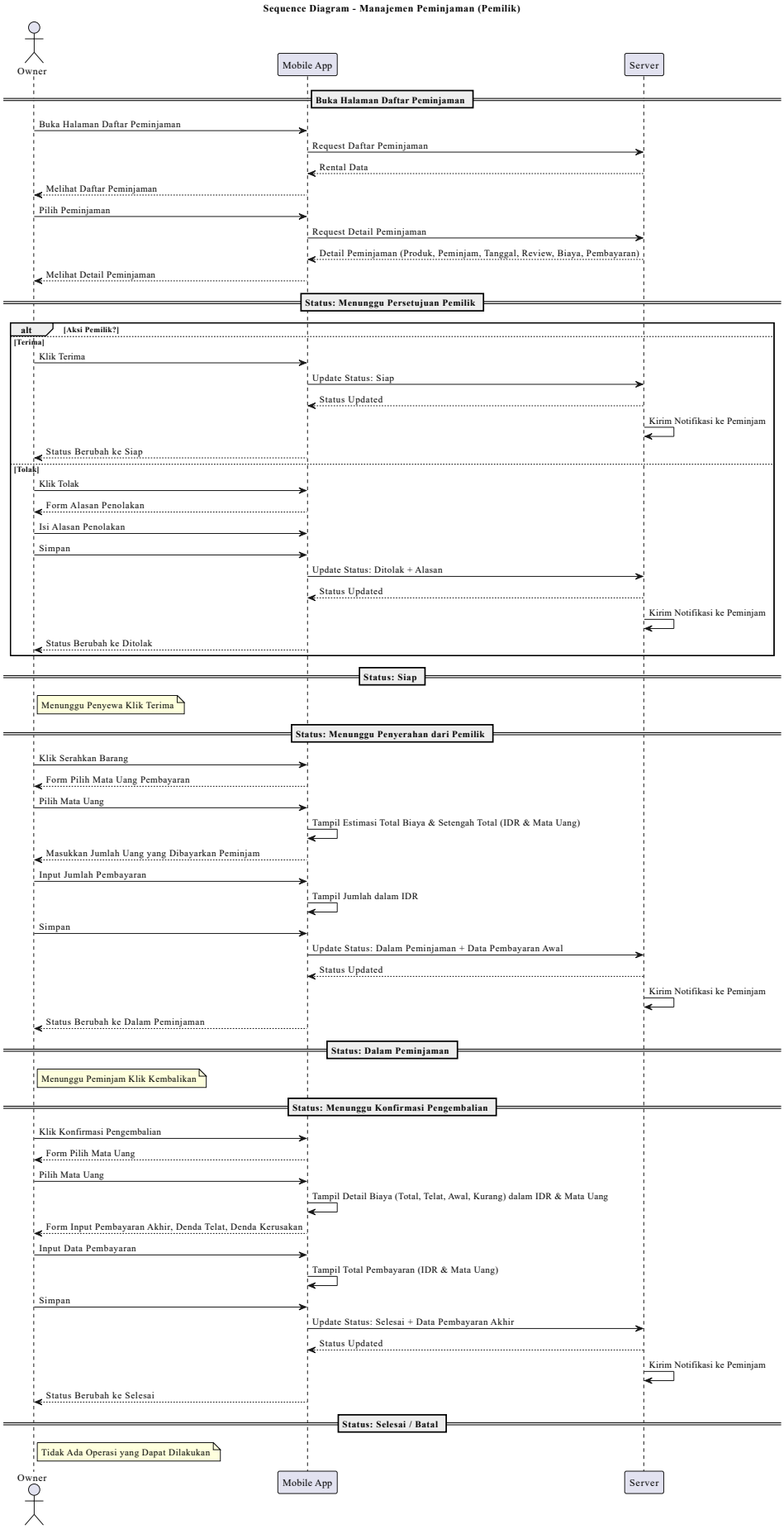
C. Manajemen Produk



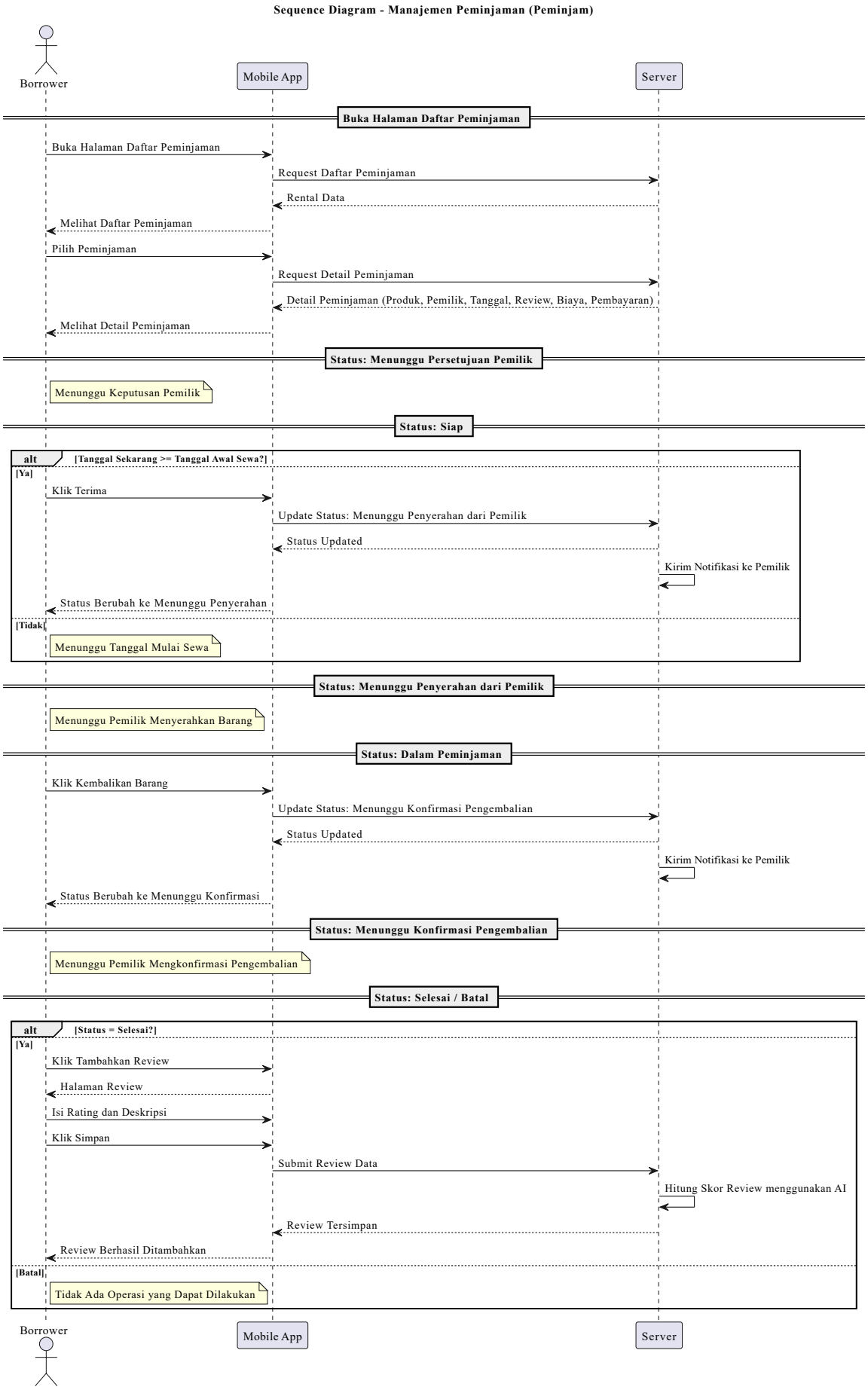
D. Pencarian dan Mulai Sewa



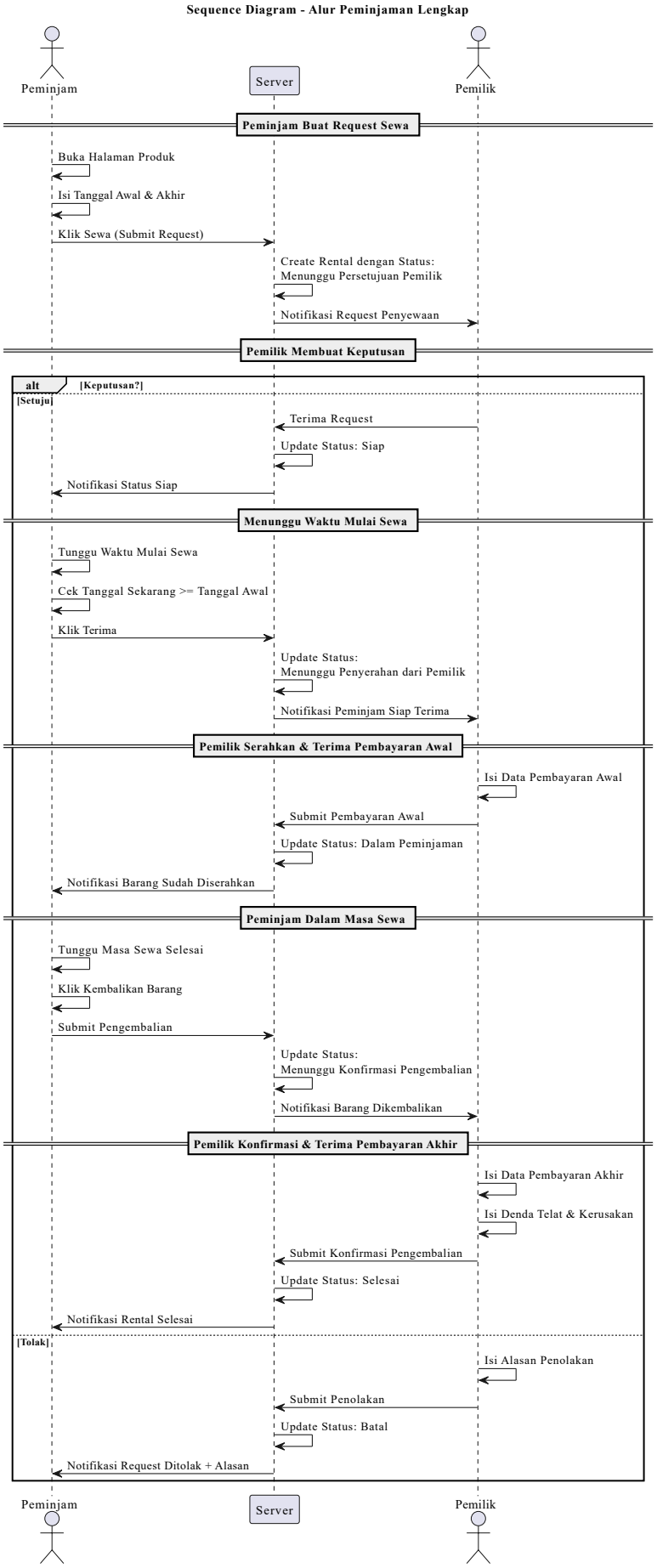
E. Manajemen Peminjaman untuk Pemilik Barang



F. Manajemen Peminjaman untuk Peminjam



G. Alur Peminjaman Secara Lengkap



2.3 Perancangan Arsitektur Sistem

Aplikasi OpenRent menggunakan arsitektur *client-server* dua tingkat:

2.3.1 Arsitektur Client (Mobile)

Aplikasi mobile dibangun menggunakan Flutter dengan pola arsitektur **BLoC (Business Logic Component)**. Lapisan aplikasi terdiri dari:

- **Presentation Layer** — Halaman (pages) dan komponen widget Flutter.
- **Business Logic Layer** — BLoC/Cubit yang mengelola state dan logika bisnis.
- **Data Layer** — Repository dan data source untuk komunikasi dengan API.

2.3.2 Arsitektur Server (Backend)

Backend dibangun menggunakan Go dengan framework Echo dan mengikuti pola **Service-Controller** dengan pemisahan ketebalan lapisan:

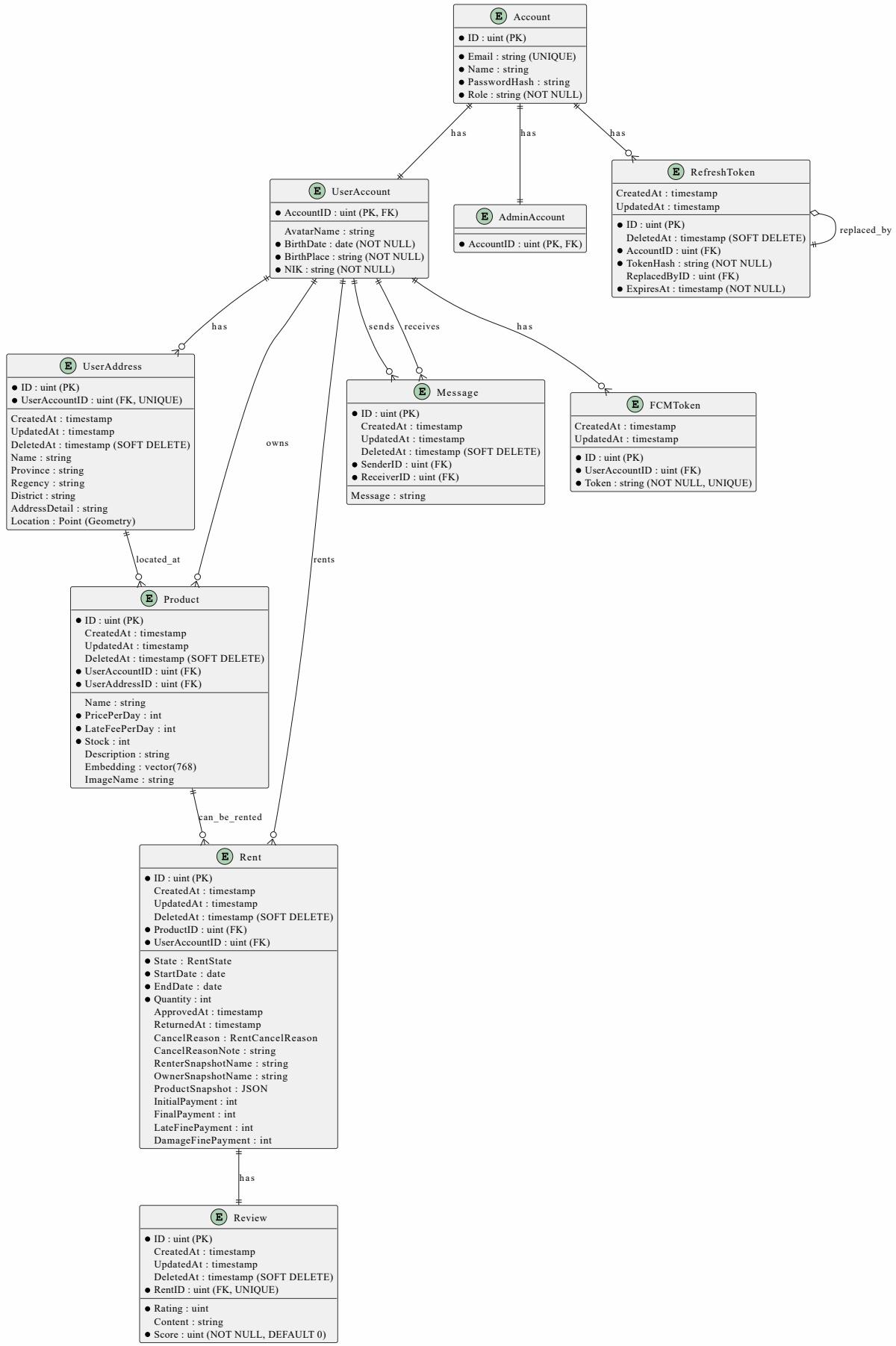
- **Controller** — Menangani request HTTP, validasi input, dan mengembalikan response.
- **Service** — Mengandung logika bisnis utama dan orchestration antar entitas.
- **Model** — Struktur data yang berinteraksi dengan database melalui ORM GORM.
- **DTO (Data Transfer Object)** — Mendefinisikan format request dan response API.
- **Core Utilities** — Helper functions untuk autentikasi, error handling, dan transformasi data.

2.3.3 Teknologi yang Digunakan

Komponen	Teknologi	Keterangan
Autentikasi	JWT	Token-based authentication
Notifikasi	Firebase Cloud Messaging	Layanan notifikasi push dari Google
Server		
Framework	Echo (Go)	Web framework untuk bahasa Go
Database	PostgreSQL	Sistem manajemen basis data relasional
Database Vector	pgvector	Eskstensi PostgresQL untuk menyimpan vector
Database Spasial	PostGIS	Ekstensi PostgresQL untuk menyimpan data spasial (lokasi)
ORM	GORM	Library ORM untuk Go
Client		
UI Framework	Flutter (Dart)	Framework UI lintas platform
State Management	BLoC	Manajemen state Flutter
HTTP Client	Retrofit dan Dio	HTTP client untuk Flutter

2.4 Perancangan Database

2.4.1 ERD Database Server



Basis data OpenRent terdiri dari entitas-entitas utama berikut:

- **Account** — Menyimpan informasi login pengguna berupa email, hash password, dan role (user/admin).
- **UserAccount** — Profil detail pengguna sebagai penyewa yang mencakup nama, tanggal lahir, NIK, dan foto avatar.
- **AdminAccount** — Profil pengguna dengan role admin untuk mengelola sistem (**tidak digunakan**).
- **UserAddress** — Alamat lengkap pengguna yang mencakup provinsi, kota, kecamatan, detail alamat, dan koordinat lokasi.
- **Product** — Data produk yang dapat disewa dengan informasi nama, harga per hari, biaya denda, stok, deskripsi, dan embedding vektor untuk pencarian.
- **Rent** — Catatan transaksi penyewaan yang melacak status, tanggal mulai/berakhir, jumlah, dan detail pembayaran.
- **Review** — Ulasan dan rating yang diberikan penyewa setelah selesai melakukan penyewaan.
- **Message** — Pesan chat antar pengguna untuk komunikasi terkait penyewaan.
- **RefreshToken** — Token untuk refresh autentikasi pengguna yang tersimpan dan dapat di-invalidate.
- **FCMToken** — Token Firebase Cloud Messaging untuk mengirim notifikasi push ke perangkat pengguna.

2.4.2 Rancangan Database Client

Refresh token dan access token disimpan menggunakan Secure Storage.

Pengturan pengguna disimpan menggunakan Hive dengan Box yang berisi data berikut:

Key	Fungsi
`needBiometric`	Menyimpan preferensi apakah pengguna ingin menggunakan autentikasi biometrik saat masuk.
`currency`	Menyimpan kode mata uang pilihan pengguna (misal: “IDR”, “USD”).
`timezone`	Menyimpan zona waktu pilihan pengguna (misal: “Asia/Jakarta”).

3 Implementasi dan Pembahasan

3.1 Lingkungan Pengembangan

Aplikasi OpenRent dikembangkan dengan menggunakan lingkungan dan alat sebagai berikut:

Alat / Tool	Versi / Spesifikasi
Flutter SDK	3.41.6
Dart SDK	3.11.4
Go	1.26
PostgreSQL	16
PostGIS	3.6.2
pgvector	0.8.2
GORM	1.31.1
IDE	Visual Studio Code dan Android Studio
Version Control	Git + GitHub
Device	Perangkat Fisik Android 16

3.2 Implementasi Backend

3.2.1 Struktur Direktori Backend

Backend OpenRent diorganisasikan menggunakan pola *feature-based*, di mana setiap fitur utama memiliki direktori tersendiri yang berisi semua komponen terkait:

```
server/
├── models/                # Struktur data dan model GORM
├── core/                  # Utilities: context, token, storage, error handling
├── ai/                    # Fitur AI: embedding dan generate
├── {fitur}/               # Direktori untuk setiap fitur, contoh:
│   ├── controller.go      # Handler HTTP dan validasi request
│   ├── service.go         # Logika bisnis utama
│   └── dto.go             # Data Transfer Objects untuk request/response
├── main.go                # Entry point aplikasi dan registrasi route
├── errors.go              # Definisi error kustom
├── go.mod                 # Manajemen dependensi
└── go.sum
```

3.2.2 Implementasi Autentikasi

Sistem autentikasi menggunakan JWT (JSON Web Token) dengan mekanisme *access token* dan *refresh token*. Proses login menghasilkan sepasang token:

- **Access Token** (short-lived): Digunakan untuk otorisasi pada setiap request API melalui header `Authorization: Bearer <token>`.

- **Refresh Token** (long-lived): Digunakan untuk mendapatkan access token baru tanpa perlu login ulang.

Untuk meningkatkan keamanan, sistem menerapkan **rotasi refresh token**, di mana setiap kali refresh token digunakan, token tersebut akan hangus dan digantikan dengan yang baru.

Middleware autentikasi (`SessionMiddleware`) memverifikasi access token pada setiap request yang dilindungi dan mengekstrak informasi sesi pengguna (ID, role) untuk digunakan di layer selanjutnya.

3.2.3 Implementasi API Endpoint

API OpenRent mengikuti prinsip RESTful. Berikut adalah daftar endpoint utama:

Metode	Endpoint	Keterangan
Autentikasi & Profil		
POST	/auth/register	Registrasi pengguna baru
POST	/auth/login	Login dan mendapatkan JWT
POST	/auth/logout	Logout dan membatalkan refresh token
GET	/auth/state	Mendapatkan profil pengguna saat ini
POST	/auth/refresh_token	Memperbarui access token dan refresh token baru
POST	/auth/avatar/presigned-url	Mendapatkan URL untuk upload avatar
POST	/auth/avatar/confirm	Konfirmasi upload avatar
POST	/auth/me/fcm	Mengirim token FCM
DELETE	/auth/me/fcm/:id	Menghapus token FCM
GET	/users/:id	Mendapatkan profil publik pengguna lain
Alamat Pengguna		
GET	/addresses	Mendapatkan semua alamat pengguna
GET	/addresses/:id	Mendapatkan detail satu alamat
POST	/addresses	Menambahkan alamat baru
PUT	/addresses/:id	Memperbarui alamat
DELETE	/addresses/:id	Menghapus alamat
Produk (Pemilik)		

GET	/me/products	Mendapatkan semua produk milik pengguna
POST	/me/products	Menambahkan produk baru
GET	/me/products/:id	Mendapatkan detail produk milik pengguna
PUT	/me/products/:id	Memperbarui produk
DELETE	/me/products/:id	Menghapus produk
POST	/me/products/:id/image/ presigned-url	Mendapatkan URL untuk upload gambar produk
POST	/me/products/:id/image/confirm	Konfirmasi upload gambar produk
Produk (Publik)		
GET	/products	Mencari dan mendapatkan semua produk
GET	/products/:id	Mendapatkan detail publik produk
Penyewaan (Penyewa)		
POST	/products/:id/rent	Mengajukan permintaan sewa
GET	/rents	Mendapatkan riwayat sewa (sebagai penyewa)
GET	/rents/:id	Mendapatkan detail sewa (sebagai penyewa)
POST	/rents/:id/receive	Konfirmasi penerimaan barang
POST	/rents/:id/return	Mengajukan pengembalian barang
Penyewaan (Pemilik)		
GET	/owner/rents	Mendapatkan daftar sewa
GET	/owner/rents/:id	Mendapatkan detail sewa
POST	/owner/rents/:id/approve	Menyetujui permintaan sewa
POST	/owner/rents/:id/reject	Menolak permintaan sewa
POST	/owner/rents/:id/handover	Konfirmasi penyerahan barang
POST	/owner/rents/:id/confirm-return	Konfirmasi barang telah kembali
POST	/owner/rents/:id/cancel	Membatalkan sewa (sebagai pemilik)
Ulasan & Chat		
POST	/rents/:id/review	Menambahkan review

DELETE	/reviews/:id	Menghapus ulasan
GET	/chats	Mendapatkan daftar percakapan
GET	/chats/:id/messages	Mendapatkan riwayat pesan dalam percakapan
POST	/chats/:id	Mengirim pesan baru
Nilai Tukar Mata Uang		
GET	/exchange-rates	Mendapatkan data tukar mata urang terhadap IDR

3.2.4 Login dan Register

Fungsi `CreateUserAccount` menangani pendaftaran pengguna baru. Fungsi ini menerima data pengguna, melakukan hashing pada password menggunakan algoritma `Argon2id` untuk keamanan, dan kemudian menyimpan data akun baru ke dalam database. Jika email sudah terdaftar, fungsi akan mengembalikan error `ErrEmailDuplicated`.

Fungsi `Login` memverifikasi kredensial pengguna. Pertama, fungsi ini mencari akun berdasarkan email. Jika ditemukan, fungsi akan membandingkan hash password yang tersimpan di database dengan password yang diberikan oleh pengguna menggunakan fungsi `argon2id.ComparePasswordAndHash`. Jika cocok, fungsi akan menghasilkan sepasang *access token* dan *refresh token* yang kemudian dikembalikan ke client.

```
type CreateUserInput struct {
  Email    string
  Name     string
  Password string
}

var argon2idParams = argon2id.Params{
  Memory:      64 * 1024,
  Iterations:  3,
  Parallelism: 2,
  SaltLength:  16,
  KeyLength:   32,
}

func (s *Service) CreateUserAccount(ctx context.Context, input CreateUserInput)
error {
  passwordHash, err := argon2id.CreateHash(input.Password, &argon2idParams)
  if err != nil {
    return err
  }
```

```
  err = gorm.G[models.Account](s.db).Create(ctx, &models.Account{
    Email:        input.Email,
    Name:         input.Name,
    PasswordHash: passwordHash,
    Role:         string(models.AccountRoleUser),
    User:         &models.UserAccount{},
  })

  if err != nil {
    if errors.Is(err, gorm.ErrDuplicatedKey) {
      return ErrEmailDuplicated
    }
  }
  return err
}

type LoginResult struct {
  ID                    uint      `json:"id"`
  Role                  string    `json:"role"`
  RefreshToken          string    `json:"refresh_token"`
  RefreshTokenExpiresAt time.Time `json:"refresh_token_expires_at"`
  AccessToken           string    `json:"access_token"`
  AccessTokenExpiresAt  time.Time `json:"access_token_expires_at"`
}

func (s *Service) Login(ctx context.Context, email string, password string)
(LoginResult, error) {
  account, err := gorm.G[models.Account](s.db).Select(
    "ID", "PasswordHash", "Email", "Role").Where(&models.Account{Email:
email}).First(ctx)

  if err != nil {
    if errors.Is(err, gorm.ErrRecordNotFound) {
      return LoginResult{}, ErrEmailNotFound
    }
    return LoginResult{}, err
  }

  match, err := argon2id.ComparePasswordAndHash(password, account.PasswordHash)

  if err != nil {
    return LoginResult{}, err
  }
```

```
  if !match {
    return LoginResult{}, ErrPasswordNotMatch
  }

  token, err := s.tokenHelper.GenerateToken(account.ID,
models.AccountRole(account.Role))

  if err != nil {
    return LoginResult{}, err
  }

  err = gorm.G[models.RefreshToken](s.db).Create(ctx, &models.RefreshToken{
    AccountID: account.ID,
    TokenHash: token.RefreshTokenHash,
    ExpiresAt: token.RefreshTokenExpiresAt,
  })

  if err != nil {
    return LoginResult{}, err
  }

  return LoginResult{
    ID:                    account.ID,
    Role:                  account.Role,
    RefreshToken:          token.RefreshToken,
    RefreshTokenExpiresAt: token.RefreshTokenExpiresAt,
    AccessToken:           token.AccessToken,
    AccessTokenExpiresAt:  token.AccessTokenExpiresAt,
  }, nil
}
```

3.2.5 JWT

Kode berikut bertanggung jawab untuk pembuatan token. Fungsi `GenerateToken` membuat sepasang *access token* dan *refresh token*. *Access token* berisi klaim standar JWT seperti ID pengguna dan role, dengan masa berlaku singkat. *Refresh token* adalah string acak yang aman secara kriptografis dan memiliki masa berlaku lebih panjang. Untuk keamanan, hanya hash dari *refresh token* yang disimpan di database, bukan token aslinya.

```
package core

import (
  "crypto/rand"
```

```
  "crypto/sha256"
  "encoding/base64"
  "encoding/hex"
  "errors"
  "openrent-server/models"
  "strconv"
  "time"

  "github.com/golang-jwt/jwt/v5"
)

var ErrInvalidSigningAlgorithm = errors.New("invalid signing algorithm")
var ErrInvalidToken = errors.New("invalid token")

type TokenHelper struct {
  key string
}

func NewTokenHelper(key string) *TokenHelper {
  return &TokenHelper{key: key}
}

const RefreshTokenLength = 32
const RefreshTokenTimeout = 30 * 24 * time.Hour

func (t *TokenHelper) GenerateRefreshToken() (string, time.Time, error) {
  bytes := make([]byte, RefreshTokenLength)
  if _, err := rand.Read(bytes); err != nil {
    return "", time.Time{}, err
  }
  return base64.RawURLEncoding.EncodeToString(bytes),
time.Now().Add(RefreshTokenTimeout), nil
}

const Timeout = time.Hour

type AccessTokenClaims struct {
  jwt.RegisteredClaims
  Scope string             `json:"scope,omitempty"`
  Role  models.AccountRole `json:"role,omitempty"`
}

func (t *TokenHelper) HashToken(token string) string {
  hash := sha256.Sum256([]byte(token))
```

```
  return hex.EncodeToString(hash[:])
}

func (t *TokenHelper) GenerateAccessToken(userId uint, role models.AccountRole)
(string, time.Time, error) {
  now := time.Now()
  expire := now.Add(Timeout)
  claims := AccessTokenClaims{
    RegisteredClaims: jwt.RegisteredClaims{
      ExpiresAt: jwt.NewNumericDate(expire),
      IssuedAt:  jwt.NewNumericDate(now),
      Subject:   strconv.FormatUint(uint64(userId), 10),
    },
    Role: role,
  }

  token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
  tokenString, err := token.SignedString([]byte(t.key))
  if err != nil {
    return "", time.Time{}, err
  }

  return tokenString, expire, nil
}

type TokenResult struct {
  AccessToken           string
  AccessTokenExpiresAt  time.Time
  RefreshToken          string
  RefreshTokenHash      string
  RefreshTokenExpiresAt time.Time
}

func (t *TokenHelper) GenerateToken(userId uint, role models.AccountRole)
(*TokenResult, error) {
  accessToken, accessTokenExpire, err := t.GenerateAccessToken(userId, role)
  if err != nil {
    return nil, err
  }

  refreshToken, refreshTokenExpire, err := t.GenerateRefreshToken()
  if err != nil {
    return nil, err
  }
```

```
  return &TokenResult{
    AccessToken:           accessToken,
    AccessTokenExpiresAt:  accessTokenExpire,
    RefreshToken:          refreshToken,
    RefreshTokenHash:      t.HashToken(refreshToken),
    RefreshTokenExpiresAt: refreshTokenExpire,
  }, nil
}

func (t *TokenHelper) VerifyAccessToken(tokenString string) (AccessTokenClaims,
error) {
  token, err := jwt.ParseWithClaims(
    tokenString,
    &AccessTokenClaims{},
    func(token *jwt.Token) (any, error) {
      if token.Method != jwt.SigningMethodHS256 {
        return nil, ErrInvalidSigningAlgorithm
      }
      return []byte(t.key), nil
    })
  if err != nil {
    return AccessTokenClaims{}, err
  }

  if !token.Valid {
    return AccessTokenClaims{}, ErrInvalidToken
  }

  claims, ok := token.Claims.(*AccessTokenClaims)
  if !ok {
    return AccessTokenClaims{}, ErrInvalidToken
  }

  return *claims, nil
}
```

3.2.6 Pencarian Berdasarkan Stok, Kata Kunci dan AI Search

```
if len(parameters.Query) > 0 {
  if parameters.DisableAISearch {
    pattern := fmt.Sprintf("%%%s%%", parameters.Query)
    q = q.Where("products.name ILIKE ? OR products.description ILIKE ?", pattern,
pattern)
```

```
  } else {
    content := fmt.Sprintf("Product search: %s", parameters.Query)
    vector, err := s.embedder.Embed(ctx, content)
    if err != nil {
      return []ResponseItemShort{}, err
    }

    q = q.Where("(products.embedding <=> ?) < 0.3", pgvector.NewVector(vector))
  }
}

if !parameters.StartDate.IsZero() && !parameters.EndDate.IsZero() {
  quntity := parameters.Quantity
  if quntity == 0 {
    quntity = 1
  }
  q = q.Joins(`
      LEFT JOIN rents ON
      rents.product_id = products.id AND
      rents.state NOT IN ? AND
      (
        (rents.start_date <= ? AND rents.end_date >= ?) OR
        (rents.end_date < CURRENT_DATE)
      )
      `,
    []models.RentState{
      models.RentStateAwaitingFinalPayment,
      models.RentStateCompleted,
      models.RentStateCancelled,
    },
    core.ConvertTimeToDate(parameters.EndDate),
    core.ConvertTimeToDate(parameters.StartDate),
  ).
    Group(`products.id, "UserAccount__Account".id,  "UserAddress".id`).
    Having("products.stock - COALESCE(SUM(rents.quantity), 0) >= ?", quntity)
}
```

3.2.7 Pengiriman Notifikasi Ke Perangkat Pengguna

Fungsi `SendNotification` bertanggung jawab untuk mengirimkan notifikasi push ke perangkat pengguna menggunakan Firebase Cloud Messaging (FCM). Prosesnya dimulai dengan mengambil semua token FCM yang terdaftar untuk `userId` tertentu dari database. Kemudian, fungsi ini mengirimkan notifikasi ke semua token tersebut secara bersamaan (multicast).

Sistem ini juga memiliki mekanisme pembersihan otomatis. Jika pengiriman ke sebuah token gagal (misalnya, karena aplikasi telah di-uninstall), token yang tidak valid tersebut akan dihapus dari database untuk mencegah upaya pengiriman yang sia-sia di masa mendatang.

```
func (s *fcmService) SendNotification(ctx context.Context, userId uint,
notification Notification) error {
  list, err := gorm.G[models.FCMToken](s.db).
    Select("id", "token").
    Where("user_account_id = ?", userId).
    Find(ctx)
  if err != nil {
    return err
  }

  if len(list) == 0 {
    return nil
  }

  tokens := lo.Map(list, func(model models.FCMToken, _ int) string {
    return model.Token
  })
  br, err := s.fcmClient.SendEachForMulticast(ctx, &messaging.MulticastMessage{
    Tokens:       tokens,
    Notification: notification.Notification,
    Data:         notification.Data,
    Android:      notification.Android,
    Webpush:      notification.Webpush,
    APNS:         notification.APNS,
    FCMOptions:   notification.FCMOptions,
  })

  if br.FailureCount > 0 {
    var failedTokenIds []uint
    for idx, resp := range br.Responses {
      if !resp.Success {
        log.Printf("failed to send notification to token %s: %v", tokens[idx],
resp.Error)
        failedTokenIds = append(failedTokenIds, list[idx].ID)
      }
    }
    if len(failedTokenIds) > 0 {
      _, err := gorm.G[models.FCMToken](s.db).
        Where("id IN ?", failedTokenIds).
```

```
        Delete(ctx)
      if err != nil {
        return err
      }
    }
  }
  if err != nil {
    return err
  }

  return nil
}
```

3.2.8 Penghitungan Skor Review Berdasarkan Skor Relevansi AI

Untuk mengurutkan review berdasarkan relevansinya, sistem menggunakan model AI generatif. Fungsi `getReviewScore` mengirimkan teks ulasan ke model AI bersama dengan sebuah *system prompt* yang mendetail.

System prompt ini menginstruksikan AI untuk bertindak sebagai ahli analisis konten dan memberikan skor "Helpfulness" (kegunaan) dari 0 hingga 100. Prompt tersebut juga menyertakan kriteria penilaian yang jelas, seperti spesifisitas, konteks transaksi, dan objektivitas, beserta bobotnya masing-masing. AI kemudian mengembalikan skor dalam format JSON, yang diekstrak oleh fungsi untuk digunakan dalam pengurutan.

```
var reviewScoreSystemPrompt = `
Anda adalah asisten ahli analisis konten untuk aplikasi persewaan (P2P Rental).
Tugas Anda adalah menilai tingkat "Helpfulness" (kegunaan) sebuah ulasan dari
pengguna. Berikan skor dari 0 hingga 100.

Kriteria Penilaian:
1. Spesifisitas (40%): Apakah ulasan menyebutkan kondisi barang secara detail?
(Contoh: "Lensa bersih, tidak ada jamur" vs "Barangnya bagus").
2. Konteks Transaksi (30%): Apakah mengulas tentang ketepatan waktu pengiriman,
keramahan pemilik, atau kemudahan proses rental?
3. Objektivitas (20%): Apakah ulasan memberikan gambaran seimbang (kelebihan dan
kekurangan)?
5. Keterbacaan (10%): Penggunaan bahasa yang jelas dan mudah dipahami.

Aturan Output:
- Output harus berupa JSON valid: {"score": number"}.
- Jangan memberikan penjelasan di luar format JSON.
- Skor 0 jika teks tidak relevan (spam/hanya karakter acak).
```

```
`

type reviewScoreResult struct {
  Score uint `json:"score"`
}

func (s *Service) getReviewScore(ctx context.Context, text string) (uint, error)
{
  result, err := s.generative.Generate(ctx, reviewScoreSystemPrompt, text,
"application/json")
  if err != nil {
    return 0, err
  }

  var resultObject reviewScoreResult
  err = json.Unmarshal([]byte(result), &resultObject)
  if err != nil {
    return 0, err
  }

  return resultObject.Score, nil
}
```

3.2.9 Pengujian Stok Pada Permintaan Sewa

Untuk mencegah *overbooking*, sistem melakukan validasi ketersediaan stok secara atomik setiap kali ada permintaan sewa baru. Logika ini menghitung jumlah stok yang sudah "dipesan" (`reservedStock`) pada rentang tanggal yang diminta.

Perhitungan ini dilakukan dengan menjumlahkan kuantitas dari semua transaksi sewa aktif (yang statusnya bukan selesai atau dibatalkan) yang periode sewanya tumpang tindih dengan periode yang diajukan. Jika jumlah stok yang diminta ditambah stok yang sudah dipesan melebihi total stok produk, sistem akan mengembalikan error `ErrStockUnavailable` dan permintaan sewa akan ditolak.

```
var reservedStock int
err = gorm.G[models.Rent](tx).
  Where("rents.product_id = ?", request.ID).
  Where(
    "rents.state NOT IN ?",
    []models.RentState{
      models.RentStateAwaitingFinalPayment,
      models.RentStateCompleted,
      models.RentStateCancelled,
    },
```

```
  ).
  Where(clause.Or(
    clause.And(
      gorm.Expr("rents.start_date <= ?",
core.ConvertTimeToDate(request.EndDate)),
      gorm.Expr("rents.end_date >= ?",
core.ConvertTimeToDate(request.StartDate)),
    ),
    gorm.Expr("rents.end_date < CURRENT_DATE"),
  )).
  Select("COALESCE(SUM(rents.quantity), 0)").
  Scan(ctx, &reservedStock)
if err != nil {
  return err
}

if request.Quantity+reservedStock > model.Stock {
  return ErrStockUnavailable
}
```

3.3 Implementasi Frontend (Mobile)

3.3.1 Struktur Direktori Client

Aplikasi Flutter diorganisasikan dengan struktur direktori berikut yang berfokus pada pemisahan antara data, logika bisnis, dan antarmuka pengguna:

```
client/
├── lib/
│   ├── data/         # Repositories dan model data
│   │   ├── remote/   # Komunikasi dengan API server
│   │   └── local/    # Akses penyimpanan lokal (secure storage dan Hive)
│   ├── bloc/         # Global BLoC (Business Logic Components)
│   ├── ui/           # Lapisan presentasi (UI)
│   │   ├── components/ # Widget yang dapat digunakan ulang
│   │   └── {fitur}/    # Halaman dan Cubit untuk setiap fitur
│   ├── biometrics/   # Logika terkait autentikasi biometrik
│   └── main.dart     # Entry point aplikasi
└── pubspec.yaml
```

3.3.2 JWT Interceptor

`AuthInterceptor` adalah komponen kunci dalam client Flutter untuk menangani autentikasi secara otomatis. Interceptor ini akan:

1. Menambahkan header `Authorization: Bearer <token>` ke setiap request API yang memerlukan otentikasi.

2. Jika sebuah request gagal dengan status 401 (Unauthorized), yang menandakan access token telah kedaluwarsa, interceptor akan secara otomatis mencoba memperbarui token menggunakan refresh token yang tersimpan.
3. Setelah mendapatkan access token baru, interceptor akan mengulang kembali request yang gagal tadi dengan token yang baru.

Mekanisme ini memastikan sesi pengguna tetap berjalan lancar tanpa perlu login ulang, bahkan setelah access token kedaluwarsa.

```
import 'package:dio/dio.dart';
import 'package:flutter/cupertino.dart';
import 'package:openrent_client/data/remote/auth.dart';
import 'package:openrent_client/data/token_storage.dart';

import 'error.dart';

class AuthInterceptor extends Interceptor {
  final Dio dio;
  final TokenStorage storage;
  final AuthService authService;

  bool _isRefreshing = false;
  final _pendingRequests =
      <({RequestOptions options, ErrorInterceptorHandler handler})>[];

  AuthInterceptor({
    required this.dio,
    required this.storage,
    required this.authService,
  });

  static final List<String> excludedPath = [
    "/auth/login",
    "/auth/refresh_token",
    "/auth/logout",
  ];

  @override
  void onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    if (excludedPath.any((path) => options.path.contains(path))) {
      return handler.next(options);
```

```
    }
    final accessToken = await storage.getAccessToken();
    if (accessToken != null) {
      options.headers['Authorization'] = 'Bearer $accessToken';
    }
    handler.next(options);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) async {
    if (err.response?.statusCode != 401) {
      return handler.next(err);
    }

    debugPrint("[AuthInterceptor] refreshing token");
    try {
      final errorData = ErrorResponse.fromJson(err.response?.data);
      if (errorData.message != "invalid token") {
        return handler.next(err);
      }
    } catch (e) {
      debugPrint('[AuthInterceptor] Failed to parse ErrorResponse: $e');
    }

    final options = err.requestOptions;

    if (excludedPath.any((path) => options.path.contains(path))) {
      return handler.next(err);
    }

    if (_isRefreshing) {
      _pendingRequests.add((options: options, handler: handler));
      return;
    }

    _isRefreshing = true;

    try {
      final refreshToken = await storage.getRefreshToken();
      debugPrint('[AuthInterceptor] Refresh token: $refreshToken');
      if (refreshToken == null) throw Exception('No refresh token');
      final newTokens = await authService.refreshToken(
        RefreshTokenRequest(refreshToken: refreshToken),
      );
```

```
      await storage.saveTokens(
        accessToken: newTokens.accessToken,
        refreshToken: newTokens.refreshToken,
      );

      await _retryRequest(options, newTokens.accessToken, handler);

      for (final pending in _pendingRequests) {
        await _retryRequest(
          pending.options,
          newTokens.accessToken,
          pending.handler,
        );
      }
    } catch (e) {
      if (e is DioException && e.response != null && e.response?.data != null) {
        try {
          final errorData = ErrorResponse.fromJson(e.response?.data);
          if (errorData.message == "invalid refresh token") {
            await storage.clearTokens();
          }
        } catch (e) {
          debugPrint('[AuthInterceptor] Failed to parse ErrorResponse: $e');
        }
      }
      final rejectError = e is DioException ? e : err;
      for (final pending in _pendingRequests) {
        pending.handler.next(rejectError);
      }
      handler.next(rejectError);
    } finally {
      _pendingRequests.clear();
      _isRefreshing = false;
    }
  }

  Future<void> _retryRequest(
    RequestOptions options,
    String newAccessToken,
    ErrorInterceptorHandler handler,
  ) async {
    options.headers['Authorization'] = 'Bearer $newAccessToken';
    try {
```

```
      final response = await dio.fetch(options);
      handler.resolve(response);
    } on DioException catch (e) {
      handler.next(e);
    }
  }
}
```

3.3.3 Autentikasi

`AuthBloc` adalah BLoC global yang bertanggung jawab untuk mengelola state autentikasi di seluruh aplikasi. BLoC ini menjadi sumber tunggal untuk mengetahui apakah pengguna sedang dalam keadaan *loading*, berhasil terautentikasi, atau gagal.

Berikut adalah alur kerjanya:

1. Saat aplikasi dimulai (`AuthBlocEventStart`), BLoC akan memeriksa apakah pengguna mengaktifkan autentikasi biometrik. Jika ya, BLoC akan meminta verifikasi biometrik terlebih dahulu.
2. Jika biometrik berhasil atau tidak diaktifkan, BLoC akan mendengarkan perubahan status dari `AuthRepository` untuk mendapatkan data pengguna saat ini.
3. Berdasarkan data dari repository, BLoC akan memancarkan state yang sesuai: `AuthStateLoading` saat proses, atau `AuthStateSuccess` dengan data profil pengguna jika berhasil (atau `null` jika tidak ada sesi).
4. Saat event `AuthBlocEventLogout` dipicu, BLoC akan menghapus sesi melalui `AuthRepository` dan menonaktifkan pengaturan biometrik.

```
import 'dart:async';

import 'package:equatable/equatable.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:openrent_client/biometrics/biometric.dart';
import 'package:openrent_client/data/auth.dart';
import 'package:openrent_client/data/resource.dart';
import 'package:openrent_client/data/settings.dart';

sealed class AuthBlocEvent {
  const AuthBlocEvent();
}

final class AuthBlocEventStart extends AuthBlocEvent {}
final class AuthBlocEventLogout extends AuthBlocEvent {}

sealed class AuthState {}
```

```
final class AuthStateLoading extends AuthState {}
final class AuthStateBiometricFailed extends AuthState {}
final class AuthStateSuccess extends AuthState {
  final AuthUserState? user;

  AuthStateSuccess(this.user);
}

class AuthBlocState extends Equatable {
  final AuthState state;

  const AuthBlocState({required this.state});

  @override
  List<Object?> get props => [state];
}

class AuthBloc extends Bloc<AuthBlocEvent, AuthBlocState> {
  final AuthRepository _authRepository;
  final SettingsRepository _settingsRepository;

  AuthBloc({required AuthRepository authRepository, required SettingsRepository
settingsRepository})
    : _settingsRepository = settingsRepository, _authRepository = authRepository,
      super(AuthBlocState(state: AuthStateLoading())) {
    on<AuthBlocEventStart>(_onStart);
    on<AuthBlocEventLogout>(_onLogout);
  }

  @override
  Future<void> close() {
    _authRepository.dispose();
    return super.close();
  }

  Future<void> _onStart(
    AuthBlocEventStart event,
    Emitter<AuthBlocState> emit,
  ) async {
    if (_settingsRepository.getNeedBiometric() && await isBiometricSupported()) {
      try {
        final result = await biometricAuthenticate(
            reason: "Authenticate for open app");
        if (!result) {
```

```
          emit(AuthBlocState(state: AuthStateBiometricFailed()));
          return;
        }
      } catch (e) {
        emit(AuthBlocState(state: AuthStateBiometricFailed()));
        return;
      }
    }
    return emit.onEach(
      _authRepository.getState(),
      onData: (state) {
        switch (state) {
          case ResourceLoading<AuthUserState?>():
            return emit(AuthBlocState(state: AuthStateLoading()));
          case ResourceSuccess<AuthUserState?>():
            return emit(AuthBlocState(state: AuthStateSuccess(state.data)));
          case ResourceError<AuthUserState?>():
            // TODO: Handle this case. This case should not happen.
            throw UnimplementedError();
        }
      },
    );
  }

  Future<void> _onLogout(
    AuthBlocEventLogout event,
    Emitter<AuthBlocState> emit,
  ) async {
    await _authRepository.logout();
    await _settingsRepository.setNeedBiometric(false);
    if (state.state is AuthStateBiometricFailed) {
      add(AuthBlocEventStart());
    }
  }
}
```

3.3.4 Mencari Alamat Terdekat dengan Pengguna

Untuk membantu pengguna memilih alamat produk, terutama saat pemilik ingin menggunakan lokasi saat ini, aplikasi mengimplementasikan fitur pencarian alamat terdekat. Kode ini mengambil lokasi pengguna saat ini (`result.data`) dan daftar alamat yang sudah ada (`addressList`).

Dengan menggunakan library `latlong2`, kode ini menghitung jarak (dalam meter) dari lokasi pengguna ke setiap alamat dalam daftar. Kemudian, kode ini melakukan iterasi untuk menemukan indeks dari alamat dengan jarak terpendek, yang kemudian dapat disarankan kepada pengguna.

```
final Distance distance = const Distance();

final distances = addressList
    .map(((item) => LatLng(item.lat, item.lng)))
    .map((item) => distance.as(LengthUnit.Meter, result.data, item))
    .toList();
int minIndex = 0;
for (int i = 1; i < distances.length; i++) {
  if (distances[i] < distances[minIndex]) {
    minIndex = i;
  }
}
```

3.3.5 Cek Stok di Tanggal atau Renttang Tanggal

Untuk memberikan umpan balik yang cepat kepada pengguna di antarmuka, aplikasi melakukan pengecekan ketersediaan stok di sisi client sebelum mengirim permintaan ke server.

- Fungsi `getAvailableStock` menghitung sisa stok pada satu hari tertentu dengan mengurangi total stok dengan jumlah barang yang sedang disewa pada hari itu.
- Fungsi `setSelectedRange` melakukan validasi untuk rentang tanggal yang dipilih pengguna. Fungsi ini menghitung jumlah stok yang terpakai pada rentang tersebut dan akan menampilkan pesan error jika stok tidak mencukupi, mencegah pengguna melanjutkan ke proses pengajuan sewa.

Logika ini memastikan pengguna mendapat informasi ketersediaan secara instan saat memilih tanggal di kalender.

```
int getAvailableStock(DateTime day) {
  final normalized = normalizeStartDate(state.timeZone, day);
  final stateSnapshot = state;
  if (stateSnapshot.data == null) return 0;

  int usedStock = 0;
  for (final rent in stateSnapshot.data!.availability) {
    if (rent.isOverdue ||
        (!rent.startDate.isAfter(normalized) &&
            !rent.endDate.isBefore(normalized))) {
      usedStock += rent.quantity;
    }
  }
  return stateSnapshot.data!.stock - usedStock;
}

void setSelectedRange(DateTime startDate, DateTime endDate) {
  final start = normalizeStartDate(state.timeZone, startDate);
```

```
  final end = normalizeEndDate(state.timeZone, endDate);

  final stateSnapshot = state;
  if (stateSnapshot.data == null) return;

  int used = 0;
  for (final rent in stateSnapshot.data!.availability) {
    if (rent.isOverdue ||
        (!rent.startDate.isAfter(end) && !rent.endDate.isBefore(start))) {
      used += rent.quantity;
    }
  }

  if (stateSnapshot.data!.stock - used <= 0) {
    emit(
      state.copyWith(
        error: .new(source: .dateRanges, message: "Tidak ada stok"),
      ),
    );
    return;
  }

  emit(state.copyWith(startDate: startDate, endDate: endDate));
}
```

3.3.6 Mini Game

Fitur mini-game “The Fragile Delivery” adalah permainan keseimbangan yang diimplementasikan menggunakan Flutter. Tujuan dari game ini adalah untuk menjaga sebuah kotak agar tetap seimbang di atas papan yang dibawa oleh karakter selama durasi waktu yang ditentukan.

Logika utama game ini adalah sebagai berikut:

- **Input Sensor:** Game memanfaatkan data dari `gyroscopeEvents` untuk mengontrol kemiringan papan dan `userAccelerometerEvents` untuk mendeteksi guncangan. Pemain kalah jika perangkat diguncang terlalu keras.
- **Fisika Sederhana:** Posisi kotak di atas papan disimulasikan menggunakan loop fisika yang berjalan setiap 16 milidetik. Kemiringan papan akan mempengaruhi kecepatan dan posisi kotak. Pemain kalah jika kotak tergelincir dari papan.
- **Sistem Skor:** Poin diberikan setiap detik berdasarkan seberapa dekat posisi kotak dengan pusat papan.
- **Visual:** Seluruh elemen game, seperti karakter, jalan, dan papan, digambar menggunakan `CustomPainter`. Animasi berjalan pada karakter dan jalan dibuat menggunakan `AnimationController` untuk memberikan ilusi pergerakan.

- **Kondisi Kemenangan:** Pemain menang dengan cara bertahan hingga waktu yang ditentukan habis. Skor akhir akan menentukan peringkat (*tier*) yang dicapai, yang memberikan hadiah berupa *frame* profil.

```
import 'dart:async';
import 'dart:math';
import 'package:flutter/material.dart';
import 'package:sensors_plus/sensors_plus.dart';

// Import kedua file komponen painter
import 'road_painter.dart';
import 'stickman_painter.dart';

class FragileDeliveryGame extends StatefulWidget {
  const FragileDeliveryGame({super.key});

  static Route<void> route() {
    return MaterialPageRoute<void>(builder: (_) => const FragileDeliveryGame());
  }

  @override
  State<FragileDeliveryGame> createState() => _FragileDeliveryGameState();
}

class _FragileDeliveryGameState extends State<FragileDeliveryGame>
    with SingleTickerProviderStateMixin {
  double plankAngle = 0.0;
  double boxX = 0.0;
  double boxVelocity = 0.0;

  final List<int> durationLevels = [10, 30, 60, 180, 300];
  int selectedDuration = 30;
  double timeElapsed = 0.0;
  int timeLeft = 30;
  int totalSum = 0;
  int lastScoredSecond = 0;

  bool isPlaying = false;
  bool isGameOver = false;
  bool isBroken = false;

  StreamSubscription<GyroscopeEvent>? _gyroSub;
  StreamSubscription<UserAccelerometerEvent>? _accelSub;
  Timer? _physicsTimer;
```

```
late AnimationController _walkAnimationController;

@override
void initState() {
  super.initState();
  _walkAnimationController = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 300),
  );
}

void startGame() {
  setState(() {
    plankAngle = 0.0;
    boxX = 0.0;
    boxVelocity = 0.0;
    timeElapsed = 0.0;
    timeLeft = selectedDuration;
    totalSum = 0;
    lastScoredSecond = 0;
    isPlaying = true;
    isGameOver = false;
    isBroken = false;
  });

  _walkAnimationController.repeat(reverse: true);

  _physicsTimer = Timer.periodic(const Duration(milliseconds: 16), (timer) {
    if (!isPlaying) return;
    setState(() {
      timeElapsed += 0.016;
      timeLeft = max(0, selectedDuration - timeElapsed.floor());

      int currentSecond = timeElapsed.floor();
      if (currentSecond > lastScoredSecond &&
          currentSecond <= selectedDuration) {
        int points = max(0, 10 - (boxX.abs() * 10).toInt());
        totalSum += points;
        lastScoredSecond = currentSecond;
      }

      boxVelocity += sin(plankAngle) * 0.02;
      boxVelocity *= 0.95;
```

```
        boxX += boxVelocity;

        if (boxX < -1.2 || boxX > 1.2) {
          loseGame('Barang Tergelincir!\nPapan terlalu miring.');
        } else if (timeElapsed >= selectedDuration) {
          winGame();
        }
      });
    });

    _gyroSub = gyroscopeEvents.listen((event) {
      if (!isPlaying) return;
      setState(() {
        plankAngle += event.y * 0.03;
        plankAngle = plankAngle.clamp(-0.8, 0.8);
      });
    });

    _accelSub = userAccelerometerEvents.listen((event) {
      if (!isPlaying) return;
      double gForce = sqrt(
        event.x * event.x + event.y * event.y + event.z * event.z,
      );
      if (gForce > 8.0) {
        loseGame('Barang Pecah!\nTerdeteksi guncangan keras.');
      }
    });
  }

  void loseGame(String reason) {
    stopGameLogic();
    setState(() {
      isPlaying = false;
      isGameOver = true;
      isBroken = true;
    });

    showResultDialog(
      'Gagal Diantar!',
      totalSum,
      'G A G A L',
      Colors.redAccent,
      reason,
    );
```

```
  }

  void winGame() {
    stopGameLogic();
    setState(() {
      isPlaying = false;
      isGameOver = true;
      timeLeft = 0;
    });

    String tierName = '';
    Color tierColor = Colors.white;
    String tierDesc = '';

    if (selectedDuration == 300 && totalSum >= 2800) {
      tierName = 'L E G E N D A R Y';
      tierColor = Colors.amberAccent;
      tierDesc = 'Akurasi Dewa (5 Menit)';
    } else if (selectedDuration >= 180 && totalSum >= 1550) {
      tierName = 'E P I C';
      tierColor = Colors.purpleAccent;
      tierDesc = 'Sangat Stabil (Min. 3 Menit)';
    } else if (selectedDuration >= 60 && totalSum >= 480) {
      tierName = 'R A R E';
      tierColor = Colors.blueAccent;
      tierDesc = 'Cukup Stabil (Min. 1 Menit)';
    } else if (selectedDuration >= 30 && totalSum >= 200) {
      tierName = 'U N C O M M O N';
      tierColor = Colors.greenAccent;
      tierDesc = 'Pemula (Min. 30 Detik)';
    } else {
      tierName = 'C O M M O N';
      tierColor = Colors.grey[400]!;
      tierDesc = 'Yang Penting Sampai';
    }

    showResultDialog(
      'Tujuan Tercapai!',
      totalSum,
      tierName,
      tierColor,
      tierDesc,
    );
  }
```

```
void showResultDialog(
  String title,
  int skorAkhir,
  String tierName,
  Color tierColor,
  String tierDesc,
) {
  showDialog(
    context: context,
    barrierDismissible: false,
    barrierColor: Colors.black.withOpacity(0.7),
    builder: (BuildContext context) {
      return AlertDialog(
        backgroundColor: Colors.grey[900]!.withOpacity(0.7),
        elevation: 0,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(20),
          side: BorderSide(color: tierColor, width: 2),
        ),
        contentPadding: const EdgeInsets.all(25),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(Icons.stars, color: tierColor, size: 60),
            const SizedBox(height: 15),
            Text(
              title,
              style: const TextStyle(
                color: Colors.white,
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 10),
            Text(
              'Total Skor Anda: $skorAkhir',
              style: const TextStyle(fontSize: 16, color: Colors.white70),
            ),
            const Padding(
              padding: EdgeInsets.symmetric(vertical: 15),
              child: Divider(color: Colors.white24),
            ),
            const Text(
```

```
              'Anda Membuka Frame Profil:',
              style: TextStyle(color: Colors.white60, fontSize: 14),
            ),
            const SizedBox(height: 5),
            Text(
              tierName,
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.w900,
                color: tierColor,
                letterSpacing: 2,
                shadows: [
                  BoxShadow(
                    color: tierColor.withOpacity(0.5),
                    blurRadius: 10,
                  ),
                ],
              ),
            ),
            const SizedBox(height: 5),
            Text(
              '($tierDesc)',
              style: TextStyle(
                color: tierColor.withOpacity(0.8),
                fontSize: 12,
              ),
            ),
          ],
        ),
        actionsAlignment: MainAxisAlignment.center,
        actions: [
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: tierColor,
              foregroundColor: Colors.black,
              padding: const EdgeInsets.symmetric(
                horizontal: 30,
                vertical: 12,
              ),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(15),
              ),
            ),
            onPressed: () {
```

```
                Navigator.of(context).pop();
                setState(() {
                  isGameOver = false;
                  isBroken = false;
                  timeLeft = selectedDuration;
                  totalSum = 0;
                  timeElapsed = 0.0;
                });
              },
              child: const Text(
                'Klaim & Selesai',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ),
          ],
        );
      },
    );
}

bool checkPrime(int number) {
  if (number <= 1) return false;
  for (int i = 2; i <= sqrt(number); i++) {
    if (number % i == 0) return false;
  }
  return true;
}

void stopGameLogic() {
  _physicsTimer?.cancel();
  _gyroSub?.cancel();
  _accelSub?.cancel();
  _walkAnimationController.stop();
}

@override
void dispose() {
  stopGameLogic();
  _walkAnimationController.dispose();
  super.dispose();
}

String formatDurationText(int seconds) {
  if (seconds < 60) return '$seconds Detik';
```

```
    return '${seconds ~/ 60} Menit';
  }

  @override
  Widget build(BuildContext context) {
    double screenWidth = MediaQuery.of(context).size.width;
    double screenHeight = MediaQuery.of(context).size.height;
    double plankWidth = screenWidth * 0.7;
    double characterCenterY = screenHeight * 0.50;
    double feetY = characterCenterY + 115;

    return Scaffold(
      appBar: AppBar(title: const Text('The Fragile Delivery')),
      body: Stack(
        children: [
          Positioned.fill(
            child: AnimatedBuilder(
              animation: _walkAnimationController,
              builder: (context, child) {
                return CustomPaint(
                  painter: PerspectiveRoadPainter(
                    timeElapsed: timeElapsed,
                    selectedDuration: selectedDuration,
                    feetY: feetY,
                  ),
                );
              },
            ),
          ),

          Positioned(
            top: 40,
            left: 0,
            right: 0,
            child: Center(
              child: Container(
                padding: const EdgeInsets.symmetric(
                  vertical: 10,
                  horizontal: 30,
                ),
                decoration: BoxDecoration(
                  color: Colors.black.withOpacity(0.6),
                  borderRadius: BorderRadius.circular(15),
                  border: Border.all(color: Colors.white24),
```

```
            ),
            child: Column(
              children: [
                Text(
                  'Sisa Jarak: $timeLeft s',
                  style: TextStyle(
                    fontSize: 24,
                    fontWeight: FontWeight.bold,
                    color: timeLeft <= 5 && isPlaying
                        ? Colors.amberAccent
                        : Colors.white,
                  ),
                ),
                Text(
                  'Total Skor: $totalSum',
                  style: const TextStyle(
                    fontSize: 16,
                    color: Colors.greenAccent,
                  ),
                ),
              ],
            ),
          ),
        ),
      ),

      Positioned(
        top: characterCenterY - 125,
        left: 0,
        right: 0,
        child: SizedBox(
          height: 250,
          width: screenWidth,
          child: Stack(
            alignment: Alignment.center,
            children: [
              Positioned(
                bottom: 10,
                child: AnimatedBuilder(
                  animation: _walkAnimationController,
                  builder: (context, child) {
                    double bobbingY = _walkAnimationController.value * 8;
                    double rockingAngle =
                        (_walkAnimationController.value - 0.5) * 0.1;
```

```
        return Transform.translate(
          offset: Offset(0, -bobbingY),
          child: Transform.rotate(
            angle: rockingAngle,
            child: CustomPaint(
              size: const Size(80, 120),
              painter: StickmanCarrierPainter(
                isPlaying
                    ? _walkAnimationController.value
                    : 0.5,
              ),
            ),
          ),
        );
      },
    ),
  ),

  Positioned(
    bottom: 125,
    child: AnimatedBuilder(
      animation: _walkAnimationController,
      builder: (context, child) {
        double bobbingY = _walkAnimationController.value * 8;
        return Transform.translate(
          offset: Offset(0, -bobbingY),
          child: child,
        );
      },
      child: Transform.rotate(
        angle: plankAngle,
        child: SizedBox(
          width: plankWidth,
          height: 80,
          child: Stack(
            alignment: Alignment.center,
            clipBehavior: Clip.none,
            children: [
              Container(
                width: plankWidth,
                height: 12,
                decoration: BoxDecoration(
                  color: Colors.orange[400],
```

```
                borderRadius: BorderRadius.circular(10),
                boxShadow: const [
                  BoxShadow(
                    color: Colors.black45,
                    blurRadius: 5,
                    offset: Offset(0, 5),
                  ),
                ],
              ),
            ),
            Align(
              alignment: Alignment(boxX, -1.0),
              child: Padding(
                padding: const EdgeInsets.only(bottom: 22.0),
                child: Container(
                  width: 50,
                  height: 50,
                  decoration: BoxDecoration(
                    color: isBroken
                        ? Colors.red[900]
                        : Colors.grey[800],
                    border: Border.all(
                      color: isBroken
                          ? Colors.redAccent
                          : Colors.amber,
                      width: 2,
                    ),
                    borderRadius: BorderRadius.circular(8),
                    boxShadow: const [
                      BoxShadow(
                        color: Colors.black45,
                        blurRadius: 4,
                        offset: Offset(2, 4),
                      ),
                    ],
                  ),
                  child: Icon(
                    isBroken
                        ? Icons.broken_image
                        : Icons.camera_alt,
                    color: isBroken
                        ? Colors.redAccent
                        : Colors.white,
                    size: 30,
```

```
                              ),
                            ),
                          ),
                        ),
                      ],
                    ),
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    ),

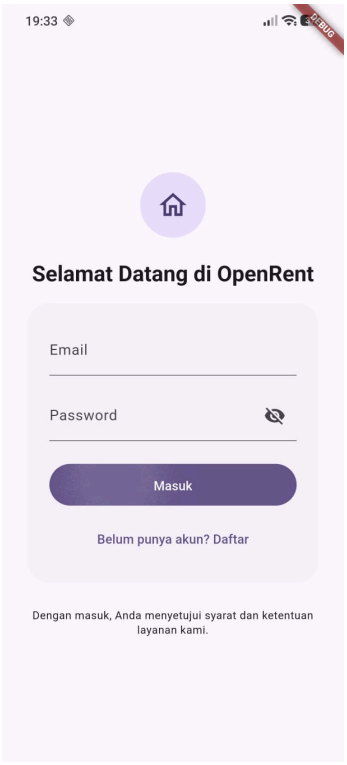
    if (!isPlaying && !isGameOver)
      Positioned(
        bottom: 40,
        left: 20,
        right: 20,
        child: Container(
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            color: Colors.black.withOpacity(0.5),
            borderRadius: BorderRadius.circular(15),
            border: Border.all(color: Colors.white12),
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Text(
                'Pilih Jarak Tempuh:',
                style: TextStyle(color: Colors.white70),
              ),
              const SizedBox(height: 10),
              Container(
                padding: const EdgeInsets.symmetric(horizontal: 15),
                decoration: BoxDecoration(
                  color: Colors.black45,
                  borderRadius: BorderRadius.circular(10),
                ),
                child: DropdownButtonHideUnderline(
                  child: DropdownButton<int>(
                    value: selectedDuration,
```

```
                dropdownColor: Colors.grey[900],
                icon: const Icon(
                  Icons.timer,
                  color: Colors.blueAccent,
                ),
                items: durationLevels.map((int value) {
                  return DropdownMenuItem<int>(
                    value: value,
                    child: Text(
                      formatDurationText(value),
                      style: const TextStyle(
                        fontWeight: FontWeight.bold,
                        color: Colors.white,
                      ),
                    ),
                  );
                }).toList(),
                onChanged: (int? newValue) {
                  if (newValue != null) {
                    setState(() {
                      selectedDuration = newValue;
                      timeLeft = newValue;
                    });
                  }
                },
              ),
            ),
          ),
          const SizedBox(height: 20),
          ElevatedButton.icon(
            style: ElevatedButton.styleFrom(
              padding: const EdgeInsets.symmetric(
                horizontal: 40,
                vertical: 15,
              ),
              backgroundColor: Colors.blueAccent,
              foregroundColor: Colors.white,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(20),
              ),
            ),
            onPressed: startGame,
            icon: const Icon(Icons.directions_run),
            label: const Text(
```

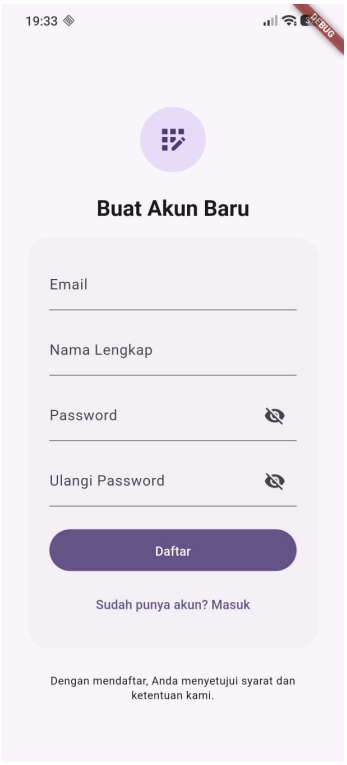
```
                    'Mulai Perjalanan',
                    style: TextStyle(fontSize: 16),
                  ),
                ),
              ],
            ),
          ),
        ),
      ],
    ),
  );
 }
}
```

3.4 Tampilan Antarmuka

3.4.0.1 Modul Autentikasi

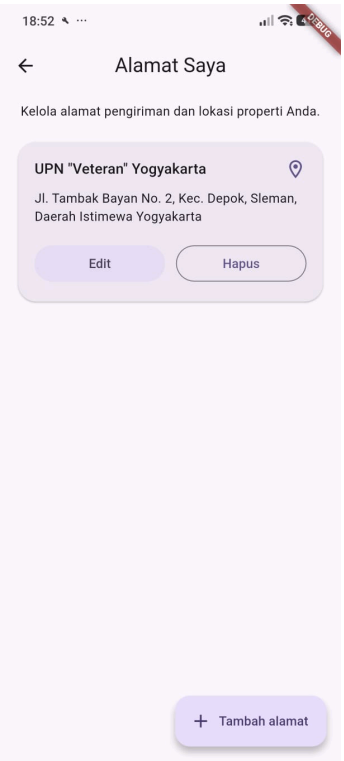


Gambar 1: Halaman Login

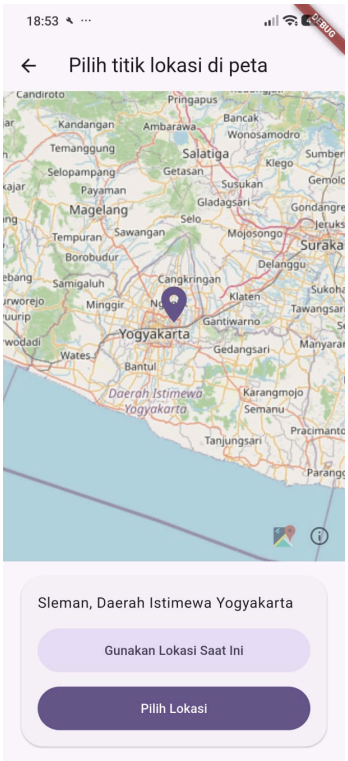


Gambar 2: Halaman Registrasi

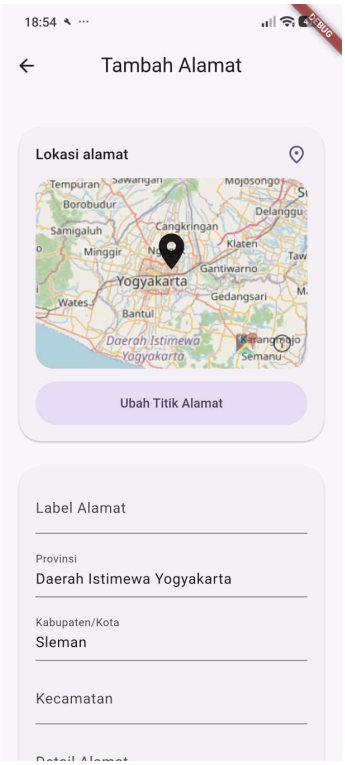
3.4.0.2 Modul Alamat



Gambar 3: Daftar Alamat

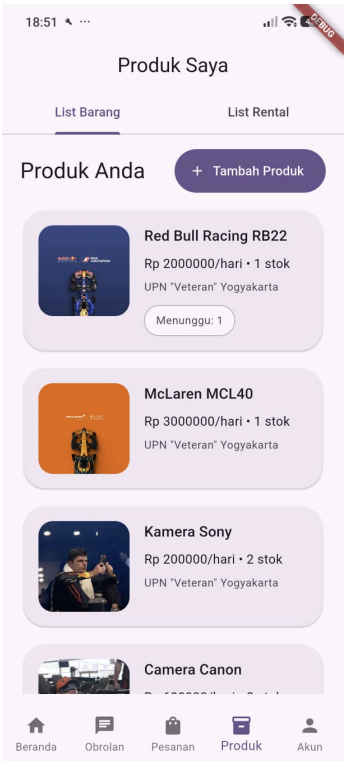


Gambar 4: Pilih Lokasi dari Peta

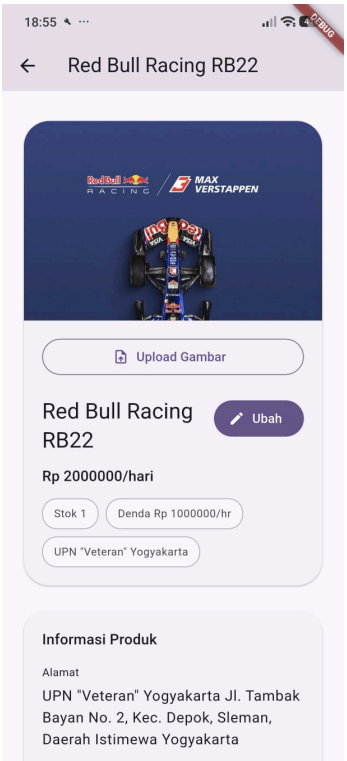


Gambar 5: Form Tambah Alamat

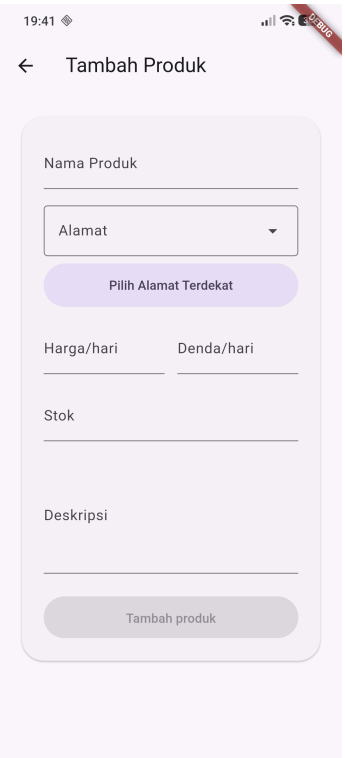
3.4.0.3 Modul Produk



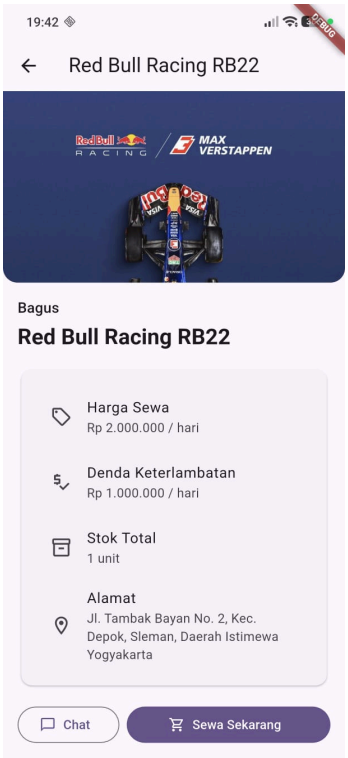
Gambar 6: Daftar Produk (Pemilik)



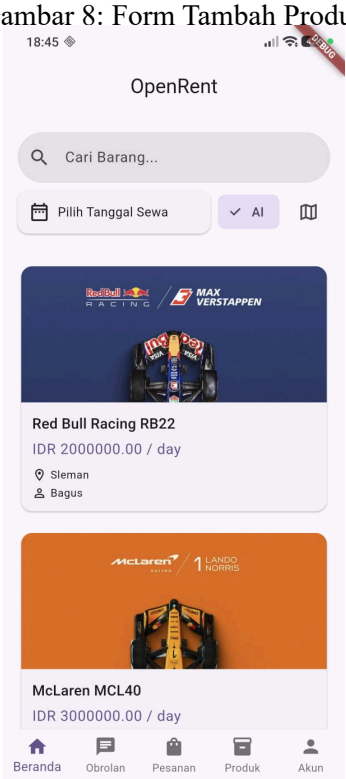
Gambar 7: Detail Produk (Pemilik)



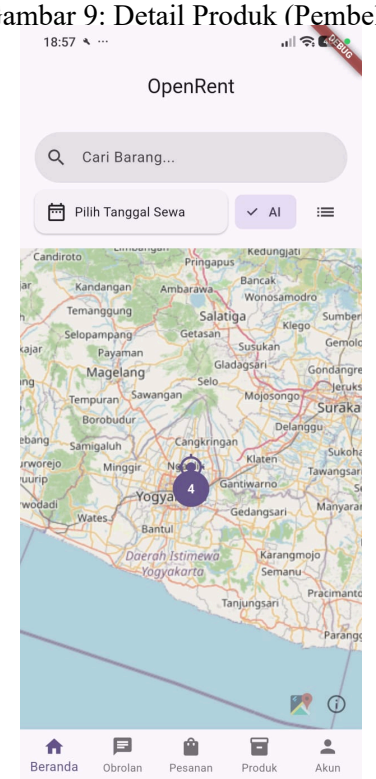
Gambar 8: Form Tambah Produk



Gambar 9: Detail Produk (Pembeli)

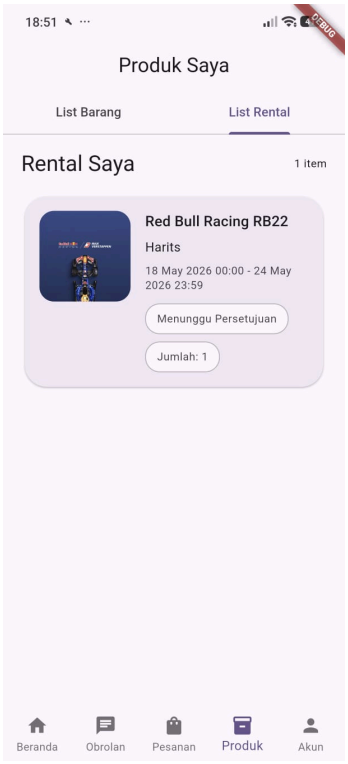


Gambar 10: Pencarian Produk

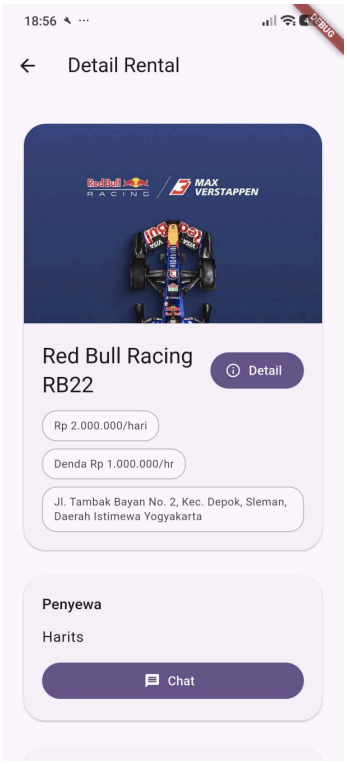


Gambar 11: Pencarian di Peta

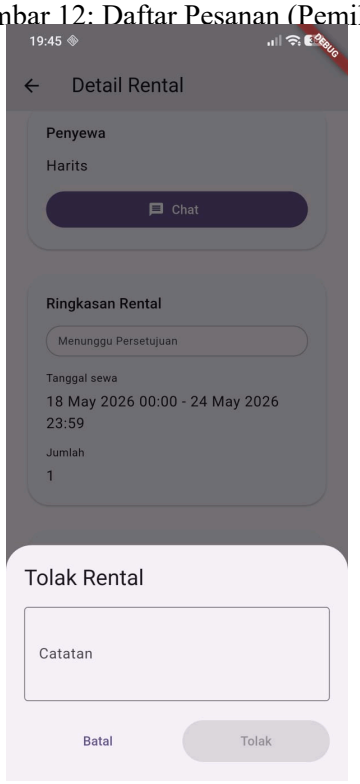
3.4.0.4 Modul Penyewaan



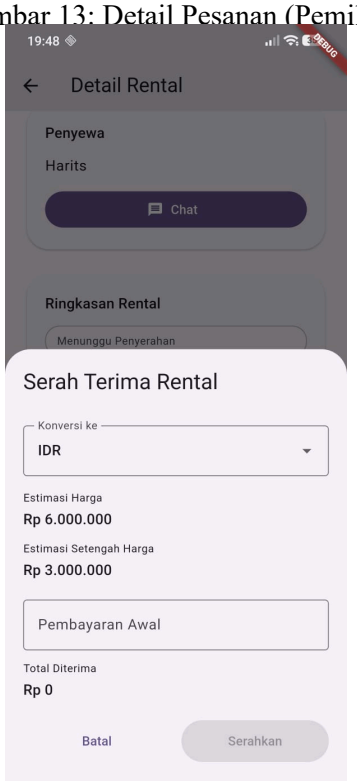
Gambar 12: Daftar Pesanan (Pemilik)



Gambar 13: Detail Pesanan (Pemilik)



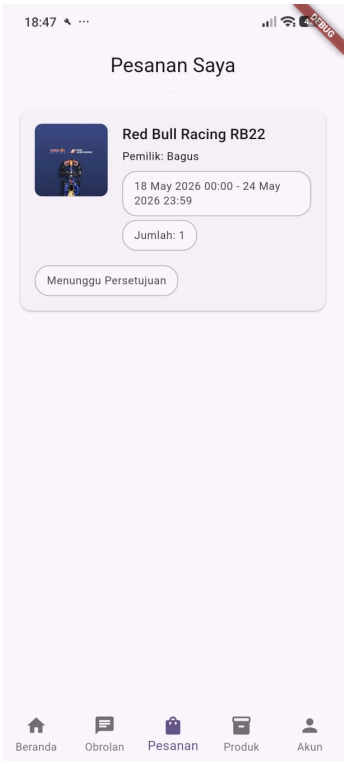
Gambar 14: Menolak Pesanan



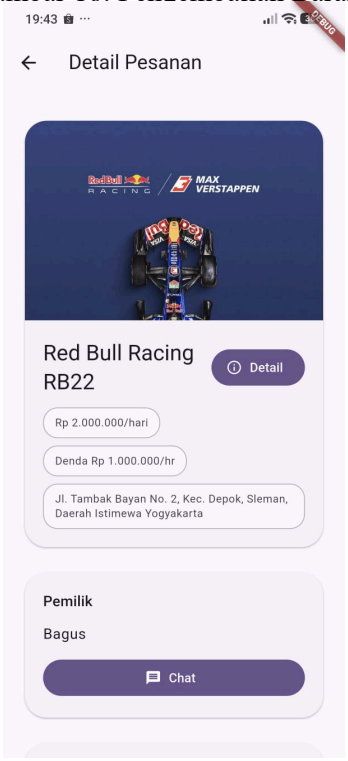
Gambar 15: Serah Terima Barang



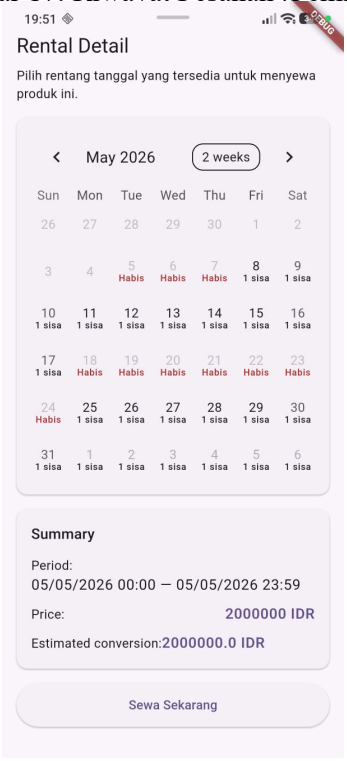
Gambar 16: Pengembalian Barang



Gambar 17: Riwayat Pesanan (Peminjam)

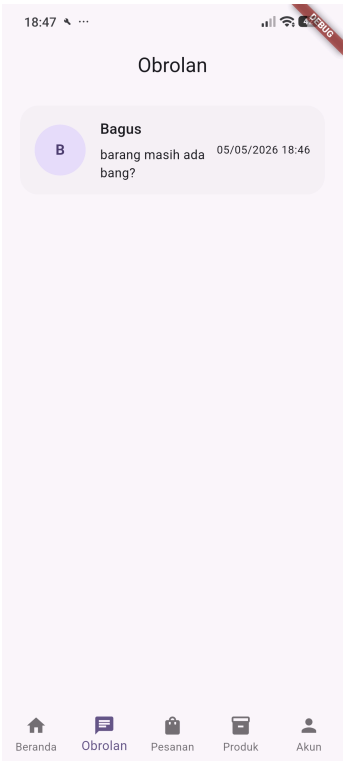


Gambar 18: Detail Pesanan (Peminjam)

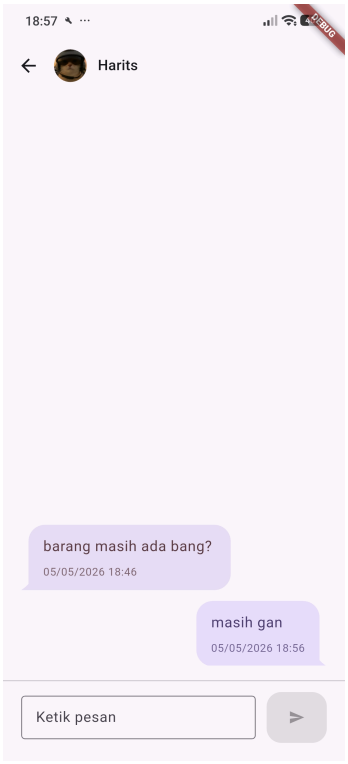


Gambar 19: Form Request Sewa

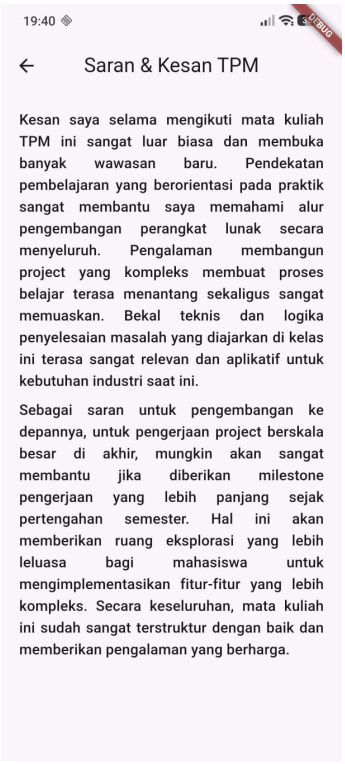
3.4.0.5 Modul Chat



Gambar 20: Daftar Percakapan

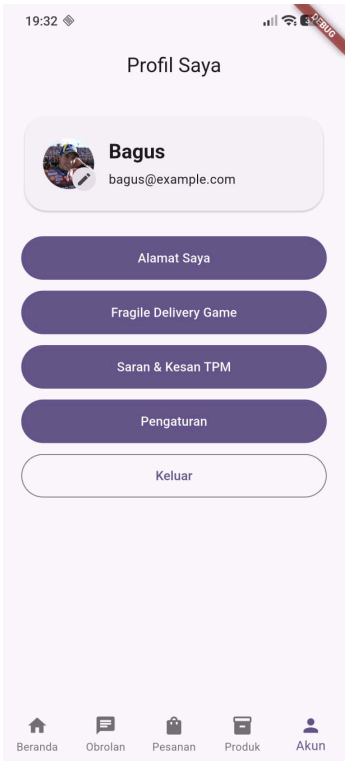


Gambar 21: Tampilan Pesan



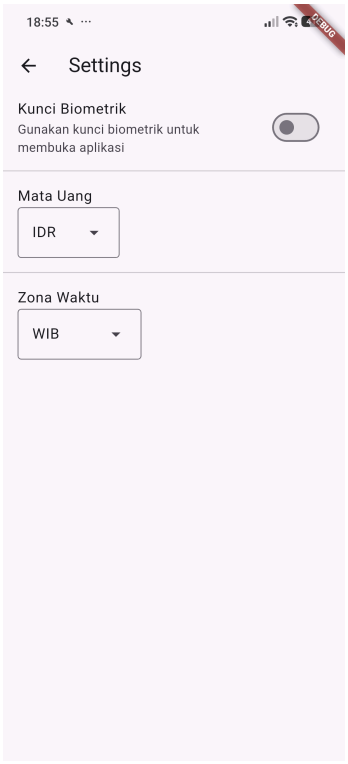
Gambar 22: Umpan Balik Pesan

3.4.0.6 Modul Profil



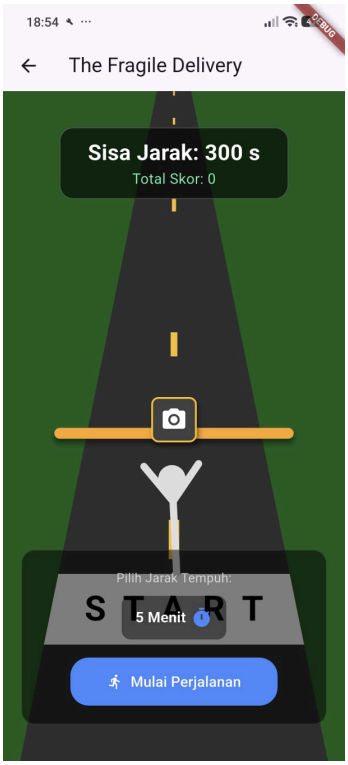
Gambar 23: Halaman Profil

3.4.0.7 Modul Pengaturan



Gambar 24: Halaman Pengaturan

3.4.0.8 Modul Mini Game



Gambar 25: Tampilan Mini Game

4 Pengujian Sistem

4.1 Metodologi Pengujian

Pengujian aplikasi OpenRent dilakukan menggunakan metode **Black-Box Testing** yang berfokus pada fungsionalitas sistem tanpa memperhatikan struktur internal kode. Setiap fitur diuji berdasarkan skenario kasus uji (*test case*) yang telah ditetapkan sebelumnya.

Pengujian dilakukan pada:

- **Perangkat:** Perangkat fisik Android 16.
- **Jaringan:** Jaringan lokal (Wi-Fi).
- **Backend:** Server lokal dalam 1 LAN.

4.2 Rencana Pengujian

4.2.1 Modul Autentikasi

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
1	Registrasi berhasil	Isi form registrasi dengan data valid, tekan tombol Daftar	Akun berhasil dibuat, pengguna diarahkan ke halaman login	✓ Lulus
2	Registrasi email duplikat	Daftarkan email yang sudah terdaftar	Muncul pesan error “Email sudah digunakan”	✓ Lulus
3	Login berhasil	Masukkan email dan password yang valid, tekan Login	Pengguna berhasil masuk dan diarahkan ke halaman utama	✓ Lulus
4	Login password salah	Masukkan email valid dengan password yang salah	Muncul pesan error “Email atau password salah”	✓ Lulus
5	Logout berhasil	Tekan tombol logout pada halaman profil	Pengguna keluar dan diarahkan ke halaman login	✓ Lulus

6	Login dengan biometrik	Aktifkan biometrik, tutup aplikasi, buka kembali	Aplikasi meminta autentikasi biometrik sebelum masuk	✓ Lulus
7	Sesi pengguna tersimpan	Login, tutup paksa aplikasi, buka kembali	Pengguna tetap dalam keadaan login tanpa harus memasukkan kredensial lagi	✓ Lulus

4.2.2 Modul Alamat

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
8	Melihat daftar alamat	Buka halaman "Alamat Saya"	Daftar alamat pengguna ditampilkan	✓ Lulus
9	Tambah alamat baru	Isi form tambah alamat, tekan Simpan	Alamat baru berhasil ditambahkan	✓ Lulus
10	Ubah alamat	Pilih alamat, ubah data, tekan Simpan	Data alamat berhasil diperbarui	✓ Lulus
11	Hapus alamat	Pilih alamat, tekan Hapus	Alamat berhasil dihapus dari daftar	✓ Lulus

4.2.3 Modul Produk

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
12	Melihat daftar produk	Buka halaman daftar produk	Daftar produk tersedia ditampilkan dengan benar	✓ Lulus
13	Melihat detail produk	Tekan salah satu produk dari daftar	Detail produk ditampilkan	✓ Lulus

			lengkap beserta foto dan review	
14	Tambah produk (Pemilik)	Isi form tambah produk dengan data valid, tekan Simpan	Produk baru berhasil ditambahkan dan muncul di daftar	✓ Lulus
15	Tambah produk dari lokasi saat ini (Pemilik)	Pada form tambah produk, pilih opsi “Gunakan Lokasi Saat Ini”	Alamat produk otomatis terisi berdasarkan koordinat GPS pengguna	✓ Lulus
16	Edit produk (Pemilik)	Ubah salah satu informasi produk, tekan Perbarui	Data produk berhasil diperbarui	✓ Lulus
17	Pencarian produk standar	Masukkan kata kunci dan rentang tanggal di bar pencarian	Hasil pencarian menampilkan produk yang relevan dan tersedia	✓ Lulus
18	Pencarian produk berbasis AI	Masukkan deskripsi naratif (misal: “kamera untuk pemula”) di pencarian	Hasil pencarian menampilkan produk yang paling sesuai secara semantik	✓ Lulus
19	Lihat rekomendasi produk	Buka detail produk	Menampilkan daftar produk serupa di bagian bawah	✓ Lulus
20	Lihat harga dalam mata uang lain	Ubah mata uang di pengaturan, buka detail produk	Estimasi harga sewa ditampilkan dalam mata uang yang dipilih	✓ Lulus

21	Urutan review berdasarkan skor relevansi AI	Buka detail produk dengan banyak review	Review dengan skor relevansi tertinggi dari AI ditampilkan paling atas	✓ Lulus

4.2.4 Modul Penyewaan

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
22	Ajukan sewa (Penyewa)	Isi form pengajuan sewa, tekan Ajukan	Pengajuan berhasil dikirim dengan status “Menunggu Persetujuan”	✓ Lulus
23	Pengajuan sewa ditolak otomatis (stok habis)	Dua pengguna mengajukan sewa pada produk & tanggal yang sama (stok 1)	Pengajuan kedua otomatis ditolak atau tidak dapat dilakukan	✓ Lulus
24	Terima pengajuan (Pemilik)	Pemilik tekan “Terima” pada pengajuan masuk	Status pengajuan berubah menjadi “Disetujui”	✓ Lulus
25	Tolak pengajuan (Pemilik)	Pemilik tekan “Tolak” pada pengajuan masuk	Status pengajuan berubah menjadi “Ditolak”	✓ Lulus
26	Konfirmasi pengambilan (Pemilik)	Pemilik mengonfirmasi pengambilan barang oleh penyewa	Status sewa berubah menjadi “Disewa”, pembayaran awal tercatat	✓ Lulus

27	Konfirmasi pengembalian (Pemilik)	Pemilik mengonfirmasi pengembalian barang	Status sewa berubah menjadi "Selesai", pembayaran akhir tercatat	✓ Lulus
28	Buat ulasan (Penyewa)	Pilih sewa yang sudah selesai, isi form ulasan, kirim	Ulasan berhasil ditambahkan pada produk terkait	✓ Lulus
29	Lihat riwayat sewa	Buka halaman "Riwayat Sewa"	Semua transaksi sewa pengguna ditampilkan dengan statusnya	✓ Lulus
30	Lihat waktu sewa sesuai zona waktu	Ubah zona waktu di pengaturan, lihat detail riwayat sewa	Waktu mulai dan selesai sewa ditampilkan sesuai zona waktu yang dipilih	✓ Lulus
31	Notifikasi perubahan status	Status sewa diubah oleh salah satu pihak	Pengguna lain menerima notifikasi push terkait perubahan status	✓ Lulus

4.2.5 Modul Chat

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
32	Mengirim pesan	Buka halaman chat dari transaksi sewa, ketik dan kirim pesan	Pesan terkirim dan diterima oleh pengguna lain	✓ Lulus
33	Menerima notifikasi pesan	Pengguna lain mengirim pesan	Menerima notifikasi push	✓ Lulus

			untuk pesan baru	

4.2.6 Modul Profil dan Pengaturan

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
34	Lihat profil	Buka halaman “Profil”	Informasi profil pengguna ditampilkan dengan benar	✓ Lulus
35	Upload Foto Profil	Klik “Ubah Foto Profil” dan pilih gambar	Perubahan foto profil berhasil disimpan	✓ Lulus
36	Ubah Pengaturan	Masuk ke halaman Pengaturan, ubah preferensi (biometrik, mata uang, zona waktu)	Perubahan berhasil disimpan dan diterapkan di aplikasi	✓ Lulus

4.2.7 Modul Mini Game

No	Skenario	Langkah Uji	Hasil yang Diharapkan	Status
37	Responsivitas sensor gerak	Buka mini game, gerakkan perangkat ke berbagai arah (miring, putar, guncang)	Objek dalam game merespons secara akurat sesuai input dari sensor giroskop dan akselerometer	✓ Lulus

4.3 Hasil Pengujian

4.3.1 Rekapitulasi Hasil Pengujian

Modul	Jumlah TC	Lulus	Persentase
Autentikasi	7	7	100%
Alamat	4	4	100%

Produk	10	10	100%
Penyewaan	10	10	100%
Chat	2	2	100%
Profil & Pengaturan	3	3	100%
Mini Game	1	1	100%
Total	**37**	**37**	**100%**

4.3.2 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang dilakukan terhadap 37 test case yang mencakup seluruh modul aplikasi OpenRent, semua skenario pengujian berhasil dilaksanakan dengan hasil yang sesuai dengan yang diharapkan. Tingkat keberhasilan pengujian mencapai 100%.

Hal ini menunjukkan bahwa aplikasi OpenRent telah berhasil mengimplementasikan seluruh fitur yang direncanakan dengan benar dan dapat berfungsi sesuai dengan spesifikasi kebutuhan fungsional yang telah ditetapkan pada tahap analisis.

5 Penutup

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Aplikasi mobile OpenRent telah berhasil dibangun menggunakan Flutter untuk sisi klien dan Go dan Echo untuk sisi server, dengan PostgreSQL sebagai basis data melalui GORM.
2. Sistem autentikasi berbasis JWT telah berhasil diimplementasikan dengan aman, menggunakan enkripsi bcrypt untuk penyimpanan kata sandi dan mekanisme validasi token pada setiap request yang memerlukan otorisasi.
3. Fitur manajemen produk untuk pemilik dan fitur pencarian serta pengajuan sewa untuk penyewa (tenant) telah berhasil diintegrasikan dalam satu platform mobile yang kohesif.
4. Antarmuka pengguna yang dibangun menggunakan Flutter menghasilkan tampilan yang responsif, intuitif, dan konsisten di berbagai ukuran layar Android.
5. Hasil pengujian Black-Box Testing menunjukkan tingkat keberhasilan 100% pada 37 test case yang mencakup seluruh modul fungsional aplikasi, membuktikan bahwa aplikasi berfungsi sesuai dengan spesifikasi kebutuhan yang telah ditetapkan.

5.2 Saran

Untuk pengembangan aplikasi OpenRent ke depannya, terdapat beberapa saran yang dapat dipertimbangkan:

5.2.1 Peningkatan Fitur

1. **Integrasi Pembayaran** — Menambahkan fitur pembayaran sewa secara online melalui integrasi dengan payment gateway seperti Midtrans atau Xendit, sehingga transaksi dapat dilakukan sepenuhnya di dalam aplikasi.

5.2.2 Peningkatan Teknis

1. **Platform iOS** — Memperluas dukungan platform ke iOS menggunakan kemampuan lintas platform Flutter, sehingga aplikasi dapat digunakan oleh pengguna iPhone dan iPad.
2. **Optimasi Performa** — Menerapkan strategi caching pada sisi klien dan implementasi pagination untuk meningkatkan performa pada dataset yang besar.
3. **CI/CD Pipeline** — Mengimplementasikan pipeline integrasi dan penerapan berkelanjutan menggunakan GitHub Actions untuk mengotomatisasi proses pengujian dan deployment.
4. **Containerization** — Menggunakan Docker untuk mengemas dan mengelola lingkungan backend, sehingga proses deployment menjadi lebih konsisten dan mudah direplikasi.
5. **Monitoring dan Logging** — Menerapkan sistem monitoring dan logging terpusat untuk memudahkan pemantauan performa dan pendeteksian masalah pada lingkungan produksi.